

Chapter 85

3D Compositing Basics

This chapter covers many of the nodes used for creating 3D composites, the tasks they perform, and how they can be combined to produce effective 3D scenes.

Contents

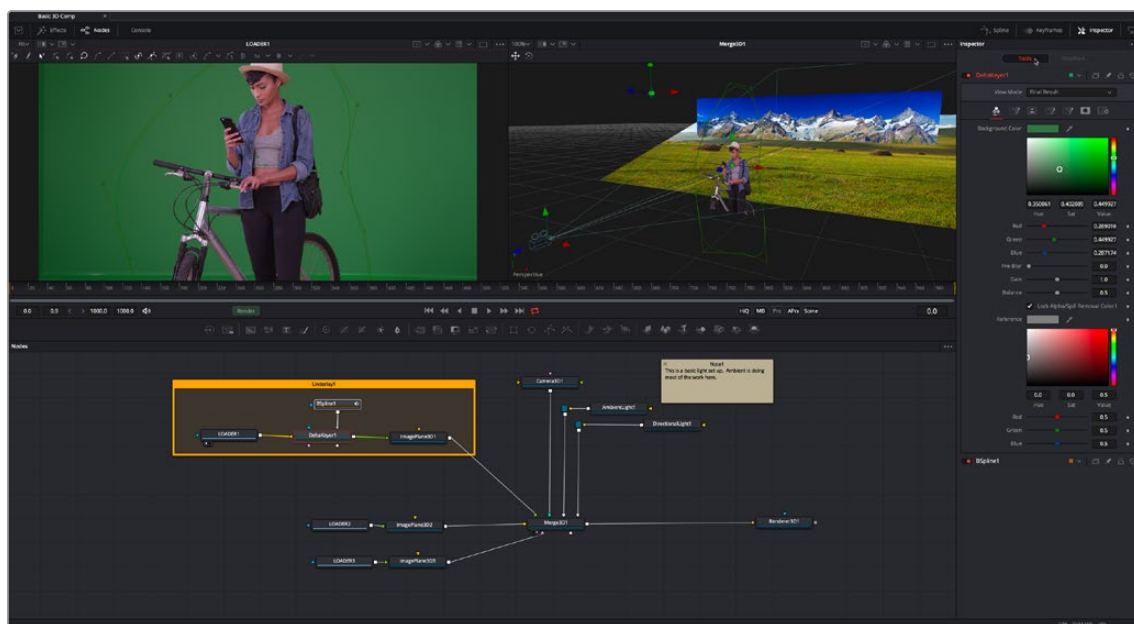
An Overview of 3D Compositing	1773	Plane of Focus and Depth of Field.....	1793
3D Compositing Fundamentals	1774	Importing Cameras.....	1794
Creating a Minimal 3D Scene.....	1774	Lighting and Shadows	1795
The Elements of a 3D Scene	1776	Enabling Lighting in the Viewer.....	1795
Geometry Nodes.....	1776	Enabling Lighting to Be Rendered.....	1795
The Merge3D Node.....	1778	Cotrolling Lighting within Each 3D Object.....	1795
The Renderer3D Node.....	1781	Lighting Types Explained.....	1796
Software vs. GPU Rendering	1782	Materials and Textures	1800
Software Renderer.....	1782	Material Components.....	1801
OpenGL Renderer.....	1782	Alpha Detail.....	1803
OpenGL UV Renderer.....	1783	Illumination Models.....	1805
Loading 3D Nodes into the Viewer	1784	Textures.....	1806
Navigating the 3D View.....	1785	Reflections and Refractions.....	1807
Transforming Cameras and Lights Using the Viewers.....	1786	Bump Maps.....	1809
Transparency Sorting	1787	Projection Mapping	1810
Material Viewer	1787	Geometry	1813
Transformations	1788	Common Visibility Parameters.....	1814
Onscreen Transform Controls.....	1789	Adding FBX Models.....	1814
Pivot.....	1790	Using Text3D.....	1816
Target.....	1790	Fog 3D and Soft Clipping	1820
Parenting	1791	Material and Object IDs	1822
Cameras	1792	World Position Pass	1822
Quickly Viewing a Scene Through a Camera.....	1793	Point Clouds	1824

An Overview of 3D Compositing

Traditional image-based compositing is a two-dimensional process. Image layers have only the amount of depth needed to define one as foreground and another as background. This is at odds with the realities of production, since all images are either captured using a live-action camera with freedom in all three dimensions, in a shot that has real depth, or have been created in a true 3D modeling and rendering application.

Within the Fusion Node Editor, you have a GPU-accelerated 3D compositing environment that includes support for imported geometry, point clouds, and particle systems for taking care of such things as:

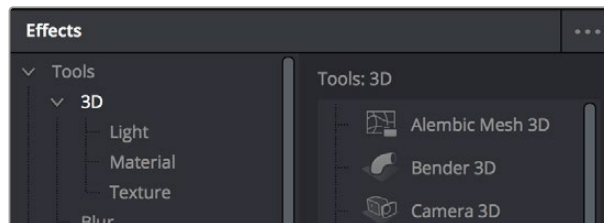
- Converting 2D images into image planes in 3D space
- Creating rough primitive geometry
- Importing mesh geometry from FBX or Alembic scenes
- Creating realistic surfaces using illumination models and shader compositing
- Rendering with realistic depth of field, motion blur, and supersampling
- Creating and using 3D particle systems
- Creating, extruding, and beveling 3D text
- Lighting and casting shadows across geometry
- 3D camera tracking
- Importing cameras, lights, and materials from 3D applications such as Maya, 3ds Max, or LightWave
- Importing matched cameras and point clouds from applications such as SynthEyes or PF Track



An example 3D scene in Fusion

3D Compositing Fundamentals

The 3D category of nodes (which includes the Light, Material, and Texture subcategories) work together to create 3D scenes. Examples are nodes that generate geometry, import geometry, modify geometry, create lights and cameras, and combine all these elements into a scene. Nearly all these nodes are collected within the 3D category of nodes found in the Effects Library.

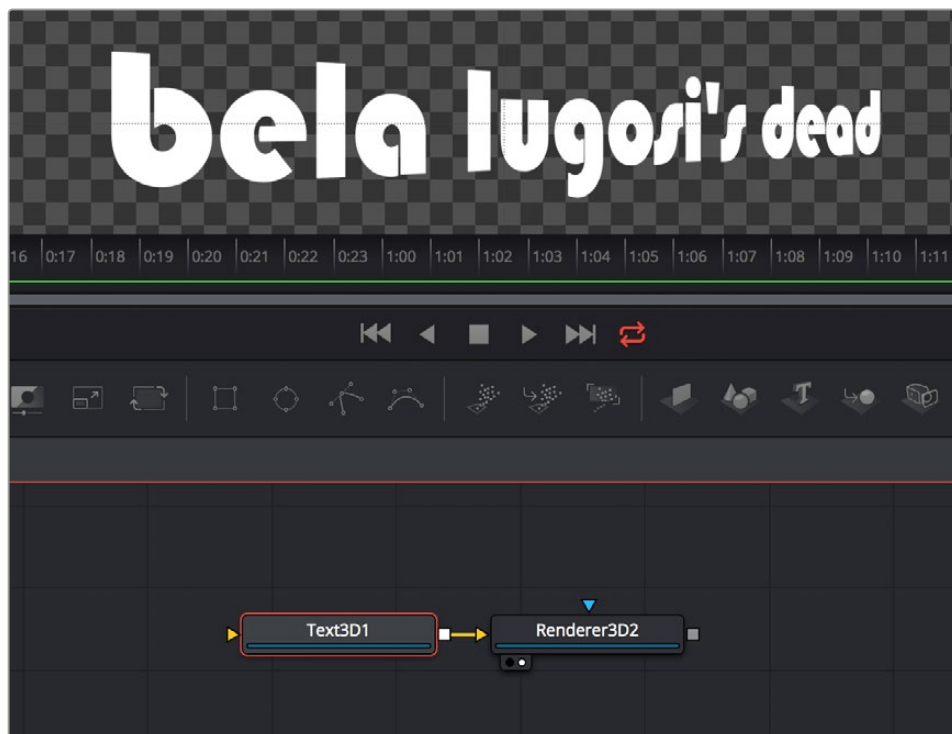


The 3D category of nodes in the Effects Library

Conveniently, at no point are you required to specify whether your overall composition is 2D or 3D, because you can seamlessly combine any number of 2D and 3D “scenes” together to create a single output. However, the nodes that create these scenes must be combined in specific ways for this to work properly.

Creating a Minimal 3D Scene

Creating a 3D scene couldn't be easier, but you need to connect the required nodes in the right way. At minimum, you need only connect a geometry node (such as a Text3D node) to a Renderer3D node to output a 2D image that can be combined with other 2D images in your composition, as seen below. However, you'll only get a simply shaded piece of geometry for your trouble, although you can color and transform it in the Inspector using controls internal to whichever geometry node you're using.

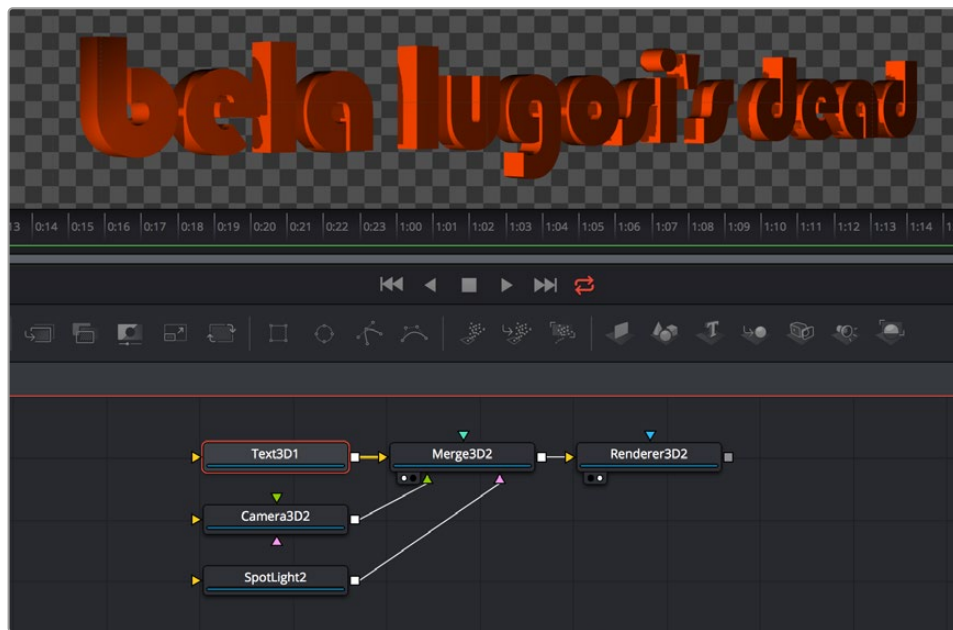


A simple 3D scene with a Text3D node connected directly to a Renderer3D node

More realistically, each 3D scene that you want to create will probably have three to five nodes to give you a better lit and framed result. These include:

- One of the available geometry nodes (such as Text3D or Image Plane 3D)
- A light node (such as DirectionalLight or SpotLight)
- A camera node
- A Merge3D node
- A Renderer3D node

All these should be connected together as seen below, with the resultantly more complex 3D scene shown below.



The same text, this time lit and framed using Text3D, Camera, and SpotLight nodes to a Merge3D node

To briefly explain how this node tree works, the geometry node (in this case Text3D) creates an object for the scene, and then the Merge3D node provides a virtual stage that combines the attached geometry with the light and camera nodes to produce a lit and framed result with highlights and shadows, while the aptly named Renderer3D node renders the resulting 3D scene to produce 2D image output that can then be merged with other 2D images in your composition.

In fact, these nodes are so important that they appear at the right of the toolbar, enabling you to quickly produce 3D scenes whenever you require. You might notice that the order of the 3D buttons on the toolbar, from left to right, corresponds to the order in which these nodes are ordinarily used. So, if you simply click on each one of these buttons from left to right, you cannot fail to create a properly assembled 3D scene, ready to work on, as seen in the previous screenshot.



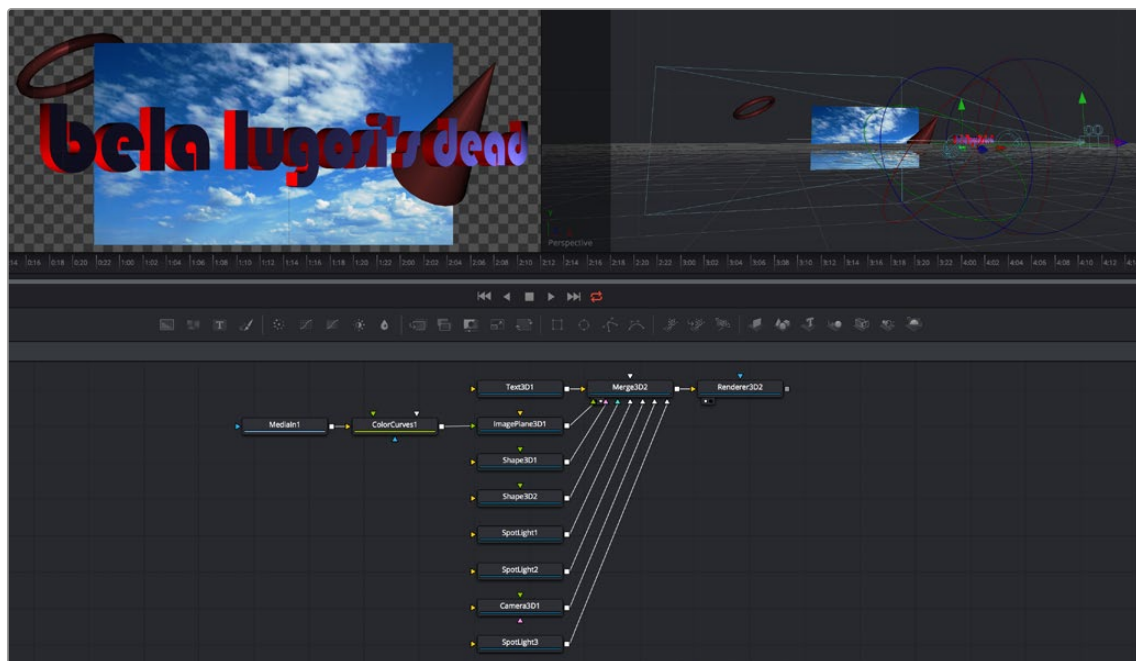
The 3D nodes available from the toolbar include the ImagePlane3D, Shape3D, Text3D, Merge3D, Camera3D, SpotLight3D, and Renderer3D nodes

The Elements of a 3D Scene

All 3D nodes can be divided into a number of categories.

Geometry Nodes

You can add 3D geometry to a composition using the ImagePlane3D node, the Shape3D node, the Cube3D node, the Text3D node, or optionally by importing a model via the FBX Mesh 3D node. Furthermore, you can add particle geometry to scenes from pEmitter nodes. You can connect these to a Merge3D node either singularly or in multiples to create sophisticated results combining multiple elements.

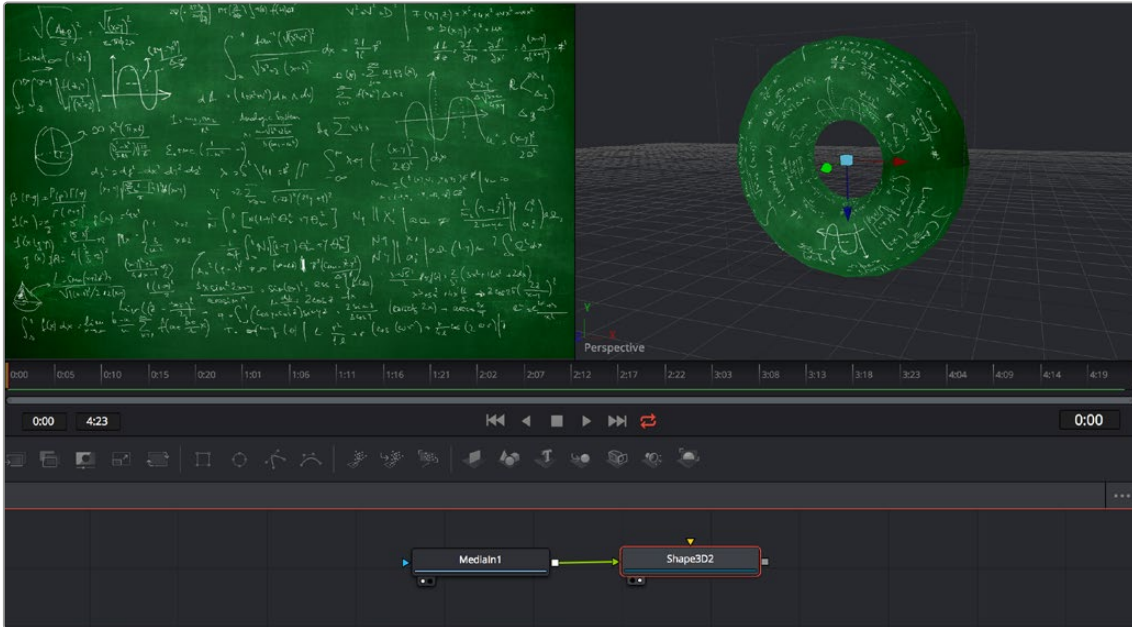


A more complex 3D scene combining several geometry nodes including the Text3D, Shape3D, and ImagePlane3D nodes

Texturing Geometry

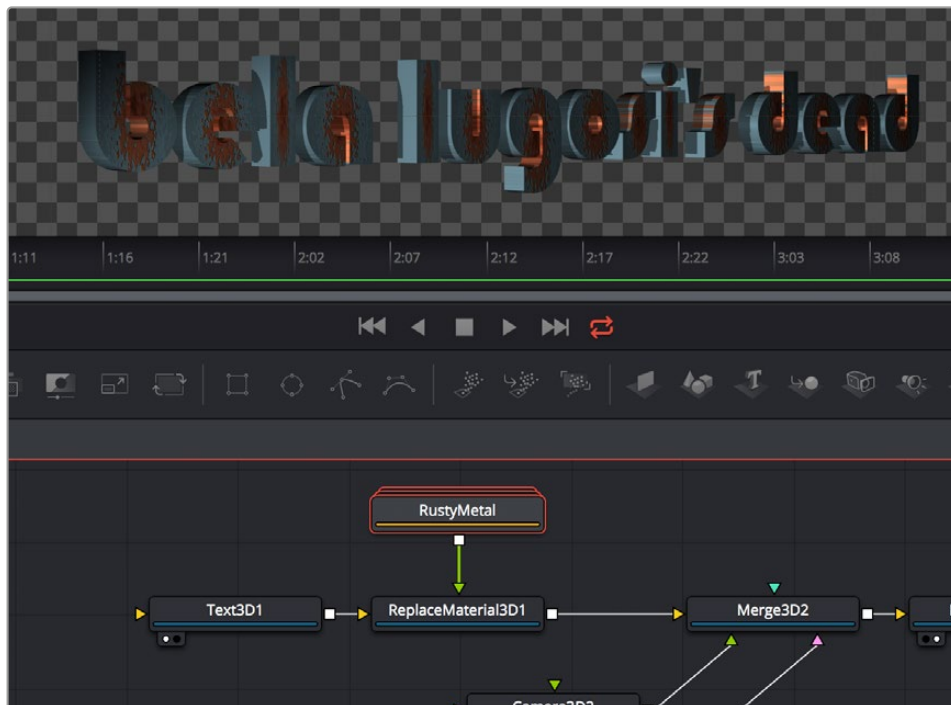
By itself, geometry nodes can only consist of a simple flat color. However, you can alter the look of 3D geometry by texturing it using clips (either still images or movies), using material nodes such as the Blinn and Phong nodes to create more sophisticated textures with combinations of 2D images and environment maps, or you can use a preset shader from the Templates > Shader bin of the Effects Library, which contains materials and texture presets that are ready to use.

If you're working with simple geometric primitives, you can texture them by connecting either an image (a still image or movie) or a shader from the Templates bin of the Effects Library directly to the material input of a Shape3D, Cube3D, or other compatible node, as shown below.



An image connected to the material input of a Shape3D node set to Torus, with the image (left), and the shaded torus (right)

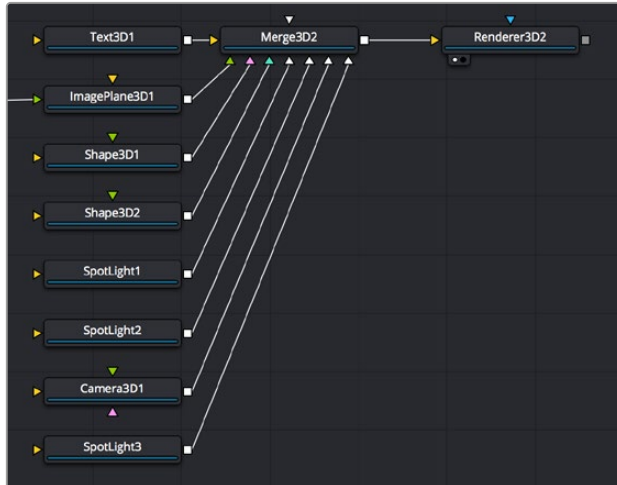
If you're shading or texturing Text3D nodes, you need to add a texture in a specific way since each node is actually a scene with individual 3D objects (the characters) working together. In the following example, the RustyMetal shader preset is applied to a Text3D node using the ReplaceMaterial3D node. The interesting thing about the ReplaceMaterial3D node is that it textures every geometric object within a scene at once, meaning that if you put a ReplaceMaterial3D node after a Text3D node, you texture every character within that node. However, if you place a ReplaceMaterial3D node after a Merge3D node, then you'll end up changing the texture of every single geometric object being combined within that Merge3D node, which is quite powerful.



The geometry created by a Text3D node is textured using a shader connected to a ReplaceMaterial3D node that's connected downstream of the object you want to shade

The Merge3D Node

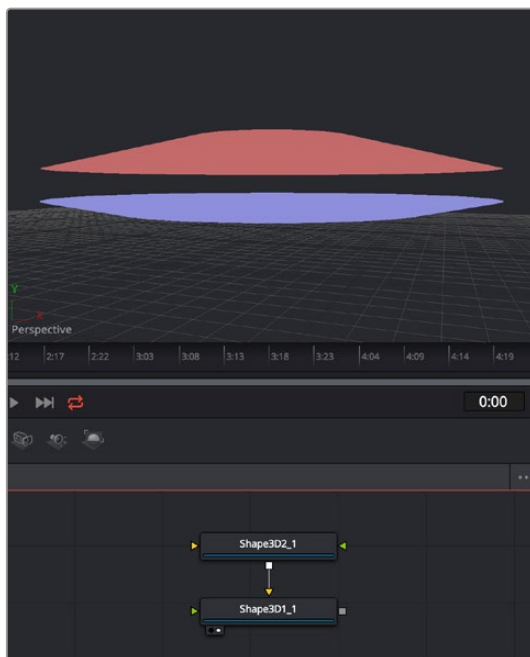
The Merge3D node combines the output of one or more 3D nodes into a single scene. Unlike the Merge2D node, the ordering of elements in the scene is not restricted to only background and foreground inputs. Instead, the Merge3D node lets you connect an unlimited number of inputs, with the resulting output combined according to each object's absolute position in 3D space.



Merging many objects together in a 3D scene using the Merge3D node

Combining Objects Directly

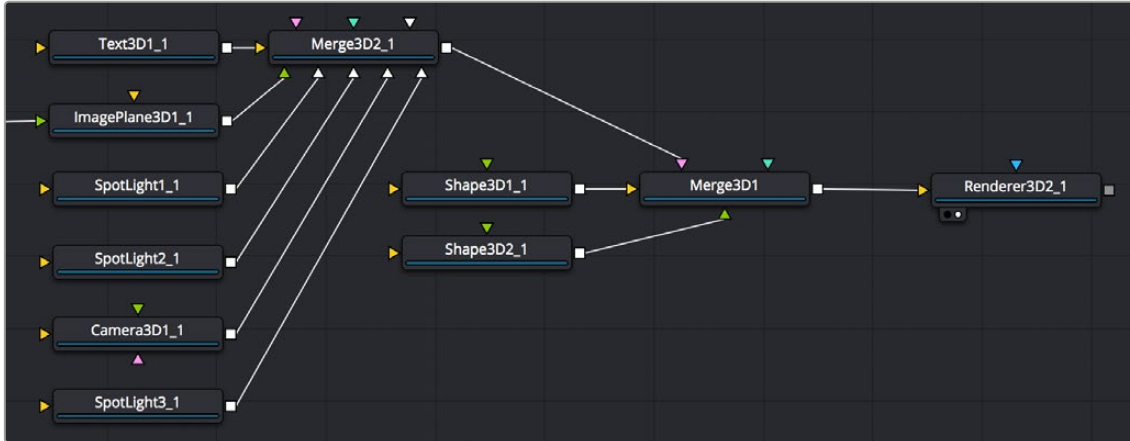
While the Merge3D node provides a structured way of combining objects, you can also combine 3D objects such as Text3D and Shape3D nodes by connecting the output of one 3D object node to the input of another, as seen in the following screenshot. When you do this, you must use each node's internal transform parameters to transform their position, size, and rotation directly, but the transform control of downstream 3D object nodes also transforms all upstream 3D object nodes. This even works for lights and the Camera3D node, giving you a fast way of combining a set of objects that always go together, which you can later connect to a Merge3D node for additional lighting and eventual connection to a Renderer3D node.



Connecting one Shape3D node to another directly to combine them. Transforming the last downstream 3D object also transforms all upstream objects; the last Shape3D node is viewed, showing both

Combining Multiple Merge3D Nodes

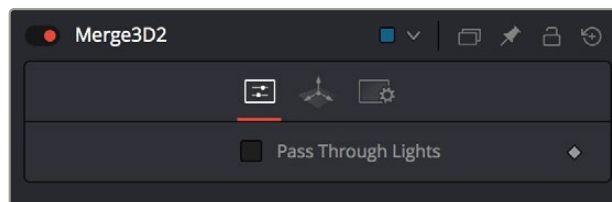
Furthermore, Merge3D nodes can be combined with other Merge3D nodes, allowing you to create composite 3D scenes made up of multiple “sub-scenes,” each put together within individual Merge3D nodes.



You can build elaborate scenes using multiple Merge3D nodes connected together

Lighting Multiple Merge3D Nodes

Once you’ve combined multiple Merge3D nodes, there’s an easy way to control how lights that are connected to upstream Merge3D nodes affect the results of other Merge3D nodes connected downstream. Each Merge3D node’s Controls tab contains a single checkbox, Pass Through Lights, which enables lighting to pass through the output of an upstream Merge3D node in order to shine onto objects connected to downstream Merge3D nodes.



You can light downstream Merge3D scenes with lights connected to upstream Merge3D scenes by turning on Pass Through Lights

This checkbox is disabled by default, which lets you light elements in one Merge3D scene without worrying about how the lighting will affect geometry attached to other Merge3D nodes further downstream. For example, you may want to apply a spotlight to brighten the wall of a building in one Merge3D node without having that spotlight spill over onto the grass or pavement at the foot of the wall modeled in another Merge3D node. In the example shown below, the left image shows how the cone and torus connected to a downstream node remain unlit by the light in an upstream node with Pass Through Lights disabled, while the right image shows how everything becomes lit when turning Pass Through Lights on.



The result of lights on the text in one Merge3D node not affecting the cone and torus added in a downstream Merge3D node (left). Turning on Pass Through Lights in the upstream Merge3D node results in those lights also illuminating the downstream shapes (right).

Transforming Merge3D Scenes

Each Merge3D node includes a Transform tab. These transform parameters adjust the position, scale, and rotation of all objects being combined within that Merge3D node together, including lighting and particles. All transformations take place around a common pivot point. This forms the basis of parenting in the 3D environment.



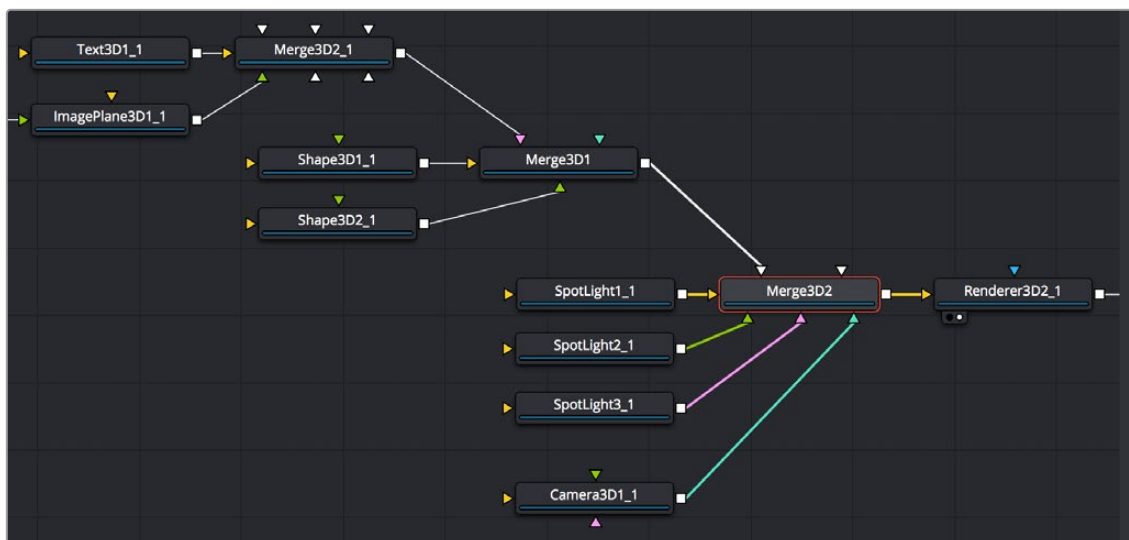
The Transform tab of a Merge3D node

If you transform a Merge3D node that's connected to other Merge3D nodes, what happens depends on which node you're transforming, an upstream node or the downstream node:

- If you transform a downstream Merge3D node, you also transform all upstream nodes connected to it as if they were all a single scene.
- If you transform an upstream Merge3D node, this has no effect on downstream Merge3D nodes, allowing you to make transforms specific to that particular node's scene.

Transforming Upstream, Lighting Downstream

When building complex scenes using multiple Merge3D nodes being combined together, it's common to use one last downstream node to combine light and camera nodes to illuminate the final scene, while leaving the upstream Merge3D nodes free for controlling object transforms and animation. This way, you can transform and animate subsets of your overall scene without worrying about accidentally altering the overall lighting scheme or cameras for that scene, unless you've specifically connected lights or cameras upstream that are meant to be attached to the geometry you're transforming.

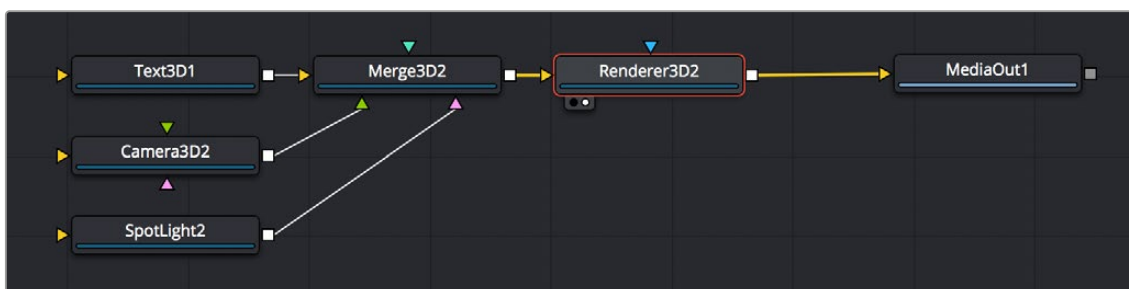


An example of a 3D scene using multiple Merge3D nodes working together; the upstream Merge3D nodes arrange the 3D objects placed within the scene, while the last Merge3D node (orange) lights and frames the scene.

The Render3D Node

Every 3D node you add outputs a complete 3D scene. This is unlike most traditional 3D modeling and animation programs, where all objects reside within a global scene environment. This means that the scenes created by a Camera 3D node and an image plane are separate until they're combined into the same scene via a Merge3D node, which itself outputs a complete 3D scene. However, this 3D scene data can neither be composited with other 2D images in your composition nor rendered out without first being rendered within the node tree using a Render3D node.

To be more specific, 3D nodes that output 3D scenes cannot be connected directly to inputs that require 2D images. For example, the output of an ImagePlane3D node cannot be connected directly to the input of a Blur node, nor can the output of a Merge3D node be directly connected to a regular Merge node. First, a Render3D node must be placed at the end of your 3D scene to render it into 2D images, which may then be composited and adjusted like any other 2D image in your composition.



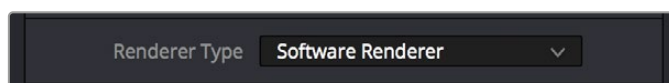
Output of a Merge3D connected to a Render3D node to output 2D image data

The **Renderer3D** uses one of the cameras in the scene (typically connected to a **Merge3D** node) to produce an image. If no camera is found, a default perspective view is used. Since this default view rarely provides a useful angle, most people build 3D scenes that include at least one camera.

The image produced by the **Renderer3D** can be any resolution with options for fields processing, color depth, and pixel aspect.

Software vs. GPU Rendering

The **Renderer3D** node lets you choose between using a software renderer or an OpenGL renderer, trading off certain aspects of rendered image quality for speed, and trading off depth of field rendering for soft shadow rendering, depending on the needs of a particular element of your composition. To choose which method of rendering to use, there's a **Renderer Type** pop-up menu in the **Controls** tab of each **Renderer3D** node's parameters in the **Inspector**. The default is **Software Renderer**.



The **Renderer Type** option in the **Controls** tab of a **Renderer3D** node

Software Renderer

The software renderer is generally used to produce the final output. While the software renderer is not the fastest method of rendering, it has twin advantages. First, the software renderer can easily handle textures much larger than one half of your GPU's maximum texture size, so if you're working with texture images larger than 8K you should choose the software renderer to obtain maximum quality.

Second, the software renderer is required to enable the rendering of "constant" and "variable" soft shadows with adjustable **Spread**, which is not supported by the OpenGL renderer. Soft shadows are more natural, and they're enabled in the **Shadows** parameters of the **Controls** tab of light nodes; you can choose **Sampling Quality** and **Softness** type, and adjust **Spread**, **Min Softness**, and **Filter Size** sliders. Additionally, the software renderer supports alpha channels in shadow maps, allowing transparency to alter shadow density.



When the **Renderer3D** node "**Renderer Type**" drop-down is set to **OpenGL Renderer**, you cannot render soft shadows or excessively large textures (left). When the **Renderer3D** node "**Renderer Type**" drop-down is set to **Software Renderer**, you can render higher-quality textures and soft shadows (right).

OpenGL Renderer

The **OpenGL** renderer takes advantage of the GPU in your computer to render the image; the textures and geometry are uploaded to the graphics hardware, and **OpenGL** shaders are used to produce the result. This can produce high-quality images that can be perfect for final rendering, and can also be potentially orders of magnitude faster than the software renderer, but it does pose some limitations

on some rendering effects, as soft shadows cannot be rendered, and the OpenGL renderer also ignores alpha channels during shadow rendering, resulting in a shadow always being cast from the entire object.

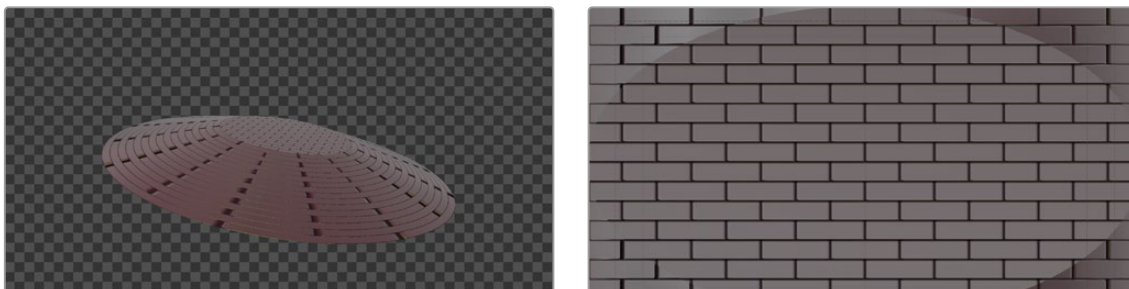
On the other hand, because of its speed, the OpenGL renderer exposes additional controls for Accumulation Effects that let you enable depth of field rendering for creating shallow-focus effects. Unfortunately, you can't have both soft shadow rendering and depth of field rendering, so you'll need to choose which is more important for any given 3D scene you render.

Don't Forget That You Can Combine Rendered Scenes in 2D

While it may seem like an insurmountable limitation that you can't output both soft shadows and depth of field using the same renderer, don't forget that you can create multiple 3D scenes each using different renderers and composite them in 2D later on. Furthermore, you can also render out auxiliary channels that can be used by 2D image processing nodes such as AmbientOcclusion, DepthBlur, and Fog to create pseudo-3D effects using the rendered images.

OpenGL UV Renderer

When you choose the OpenGL UV Renderer option, a `Renderer3D` node outputs an “unwrapped” version of the textures applied to upstream objects, at the resolution specified within the Image tab of that `Renderer3D` node.



A normally rendered 3D scene (left), and the same scene rendered using the OpenGL UV Renderer mode of the `Renderer3D` node (right)

This specially output image is used for baking out texture projections or materials to a texture map for one of two reasons:

- Baking out projections can speed up a render.
- Baking out projections lets you modify the texture using other 2D nodes within your composition, or even using third-party paint applications (if you output this image in isolation as a graphics file) prior to applying it back onto the geometry.

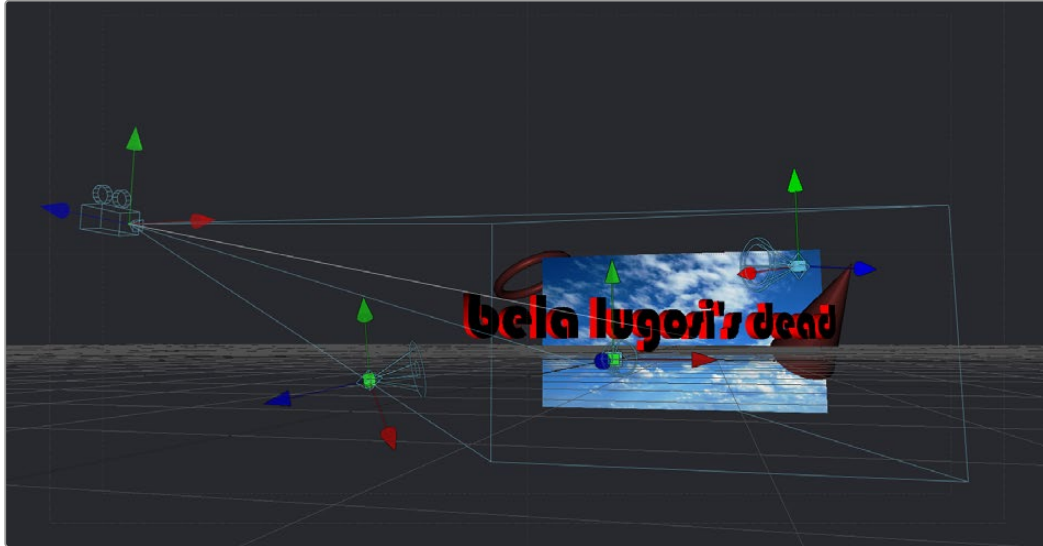
Suppose, for instance, that you have a scene on a street corner, and there's a shop sign with a phone number on it, but you want to change the numbers. If you track the scene and have standing geometry for the sign, you can project the footage onto it, do a UV render, switch the numbers around with a `Paint` node, and then apply that back to the mesh with a `Texture2D`.

The UV renderer can also be used for retouching textures. You can combine multiple DSLR still shots of a location, project all those onto the mesh, UV render it out, and then retouch the seams and apply it back to the mesh.

You could project tracked footage of a road with cars on it, UV render out the projection from the geometry, do a temporal median filter on the frames, and then map a “clean” roadway back down.

Loading 3D Nodes into the Viewer

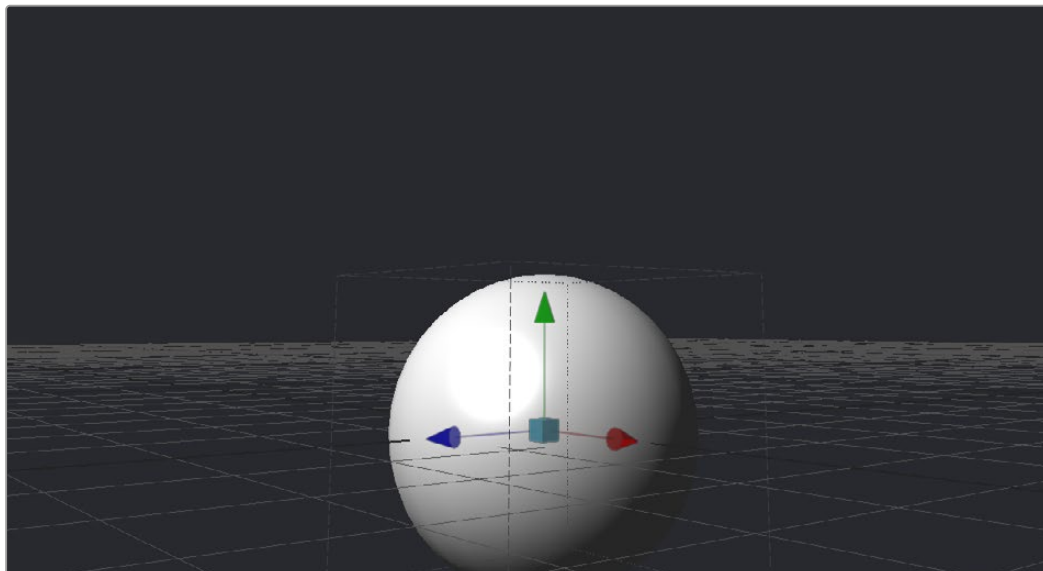
When you load a 3D node into the viewer, it switches to a 3D Viewer, which lets you pan, zoom, and rotate the scene in 3D, making it easy to make adjustments in three dimensions.



The 3D Viewer

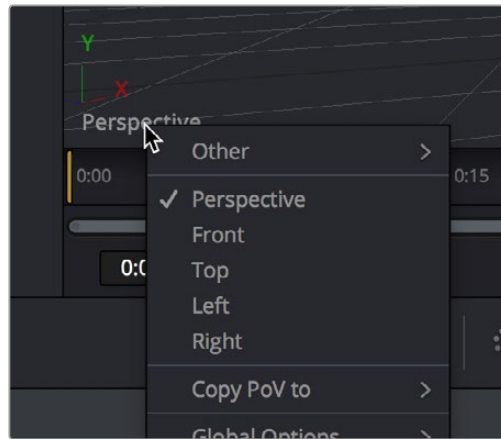
The interactive 3D Viewer is highly dependent on the computer's graphics hardware, relying on support from OpenGL. The amount of onboard memory, as well as the speed and features of your workstation's GPU, make a huge difference in the speed and capabilities of the 3D Viewer.

Displaying a node with a 3D output in any viewer will switch the display type to a 3D Viewer. Initially, the contents of the scene will be displayed through a default perspective view.



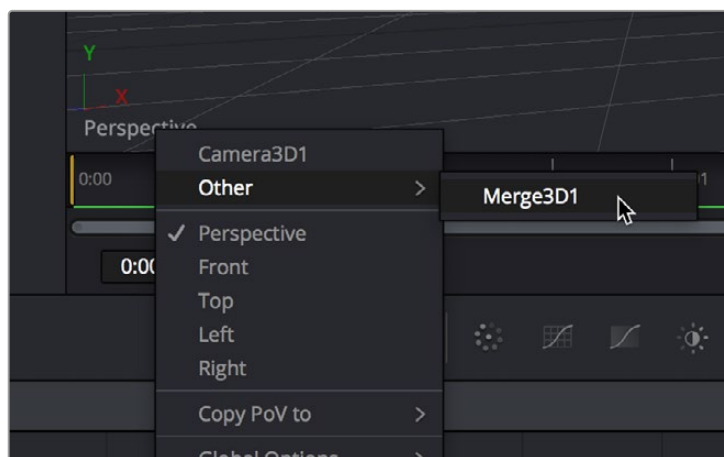
A 3D Viewer's default perspective view

To change the viewpoint, right-click in the viewer and choose the desired viewpoint from the ones listed in the Camera submenu. A shortcut to the Camera submenu is to right-click on the axis label displayed in the bottom corner of the viewer.



Right-click the Axis label of the viewer to change the viewpoint

In addition to the usual Perspective, Front, Top, Left, and Right viewpoints, if there are cameras and lights present in the scene as potential viewpoints, those are shown as well. It's even possible to display the scene from the viewpoint of a Merge3D or Transform3D by selecting it from the contextual menu's Camera > Other submenu. Being able to move around the scene and see it from different viewpoints can help with the positioning, alignment, and lighting, as well as other aspects of your composite.



The Perspective drop-down menu also shows cameras, lights, and Merge3D and Transform3D nodes you can switch to.

Navigating the 3D View

For the most part, panning and scaling of the 3D Viewer uses the same controls as the 2D Viewer.

For more information about the options available in the 3D Viewer, see *Chapter 69, "Using Viewers."* in the DaVinci Resolve Reference Manual or Chapter 7 in the Fusion Reference Manual.

To pan in a 3D Viewer, do the following:

- Hold the middle mouse button and drag in the viewer.

To dolly (zoom) in the 3D Viewer, do one of the following:

- Hold down the middle and left mouse buttons and drag left or right in the viewer.
- Hold down the Command key and use your pointing device's scroll control.

To rotate around the 3D Viewer, do the following:

- Hold down the Option key and middle-button-drag left and right in the viewer.

If you want to frame certain objects in the viewer:

- 1 Select the viewer you want to work in.
- 2 Do one of the following:
 - Press Shift-F to Fit all objects in the viewer.
 - Press F to Fit to selection (or Fit All if nothing is selected).
 - Press D to Rotate the viewer to look at the center of the currently selected object without moving the viewer's position.

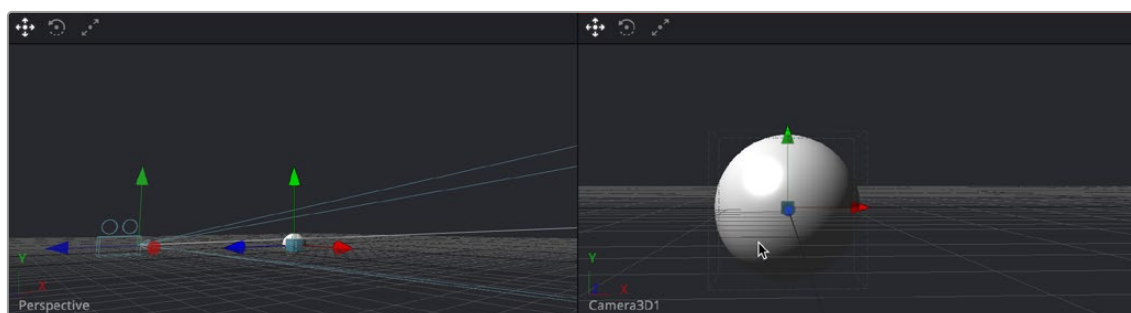
Furthermore, selecting a 3D node in the Node Editor also selects the associated object in the 3D Viewer.

Transforming Cameras and Lights Using the Viewers

When the viewer is set to look through a 3D object in the scene, such as a camera or spotlight, the usual controls for panning and rotating the viewer will now directly affect the position of the camera or spotlight you're viewing through. Here's an example.

To adjust a camera's position when looking through it in a viewer:

- 1 Right-click the viewpoint label, and choose a camera from the contextual menu. (Optional) If you're in dual-viewer mode, you can load the camera you've selected in one viewer into the other viewer to see its position as you work.
- 2 Move the pointer into the viewer that's displaying the camera's viewpoint.
- 3 Hold the middle and left mouse buttons down and drag to zoom the viewer, or middle-click-drag to pan the viewer, or option-middle-click-drag to rotate the viewer, all while also moving the camera.



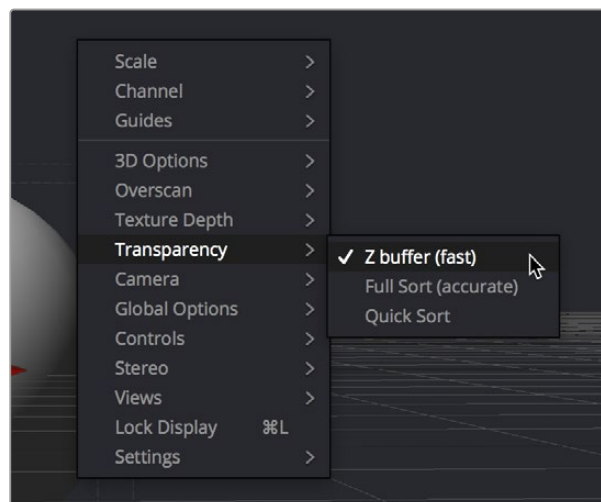
When a viewer is set to display the view of a camera or light, panning, zooming, or rotating the viewer (seen at right) actually transforms the camera or light you're viewing through (seen at left)

It is even possible to view the scene from the perspective of a Merge3D or Transform3D node by selecting the object from the Camera > Others menu. The same transform techniques will then move the position of the object. This can be helpful when you are trying to orient an object in a certain direction.

Transparency Sorting

While generally the order of geometry in a 3D scene is determined by the Z-position of each object, sorting every face of every object in a large scene can take an enormous amount of time. To provide the best possible performance, a Fast Sorting mode is used in the OpenGL renderer and viewers. This is set by right-clicking in the viewer and choosing Transparency > Z-buffer. While this approach is much faster than a full sort, when objects in the scene are partially transparent it can also produce incorrect results.

The Sorted (Accurate) mode can be used to perform a more accurate sort at the expense of performance. This mode is selected from the Transparency menu of the viewer's contextual menu. The Renderer3D also presents a Transparency menu when the Renderer Type is set to OpenGL. Sorted mode does not support shadows in OpenGL. The software renderer always uses the Sorted (Accurate) method.

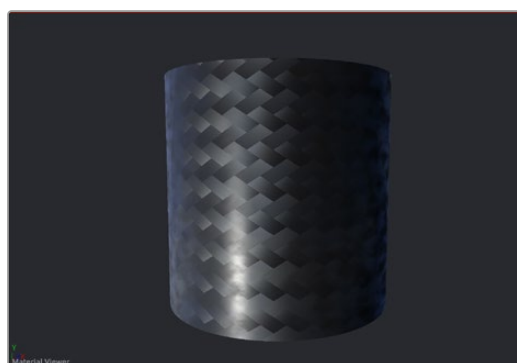


Transparency Sorting in the viewer contextual menu

The basic rule is when a scene contains overlapping transparency, use the Full/Quick Sort modes, and otherwise use the Z-buffer (Fast). If the Full Sort method is too slow, try switching back to Z-buffer (Fast).

Material Viewer

When you view a node that comes from the 3D > Material category of nodes in the Effects Library, the viewer automatically switches to display a Material Viewer. This Material Viewer allows you to preview the material applied to a lit 3D sphere rendered with OpenGL by default.



The Material Viewer mode of the viewer

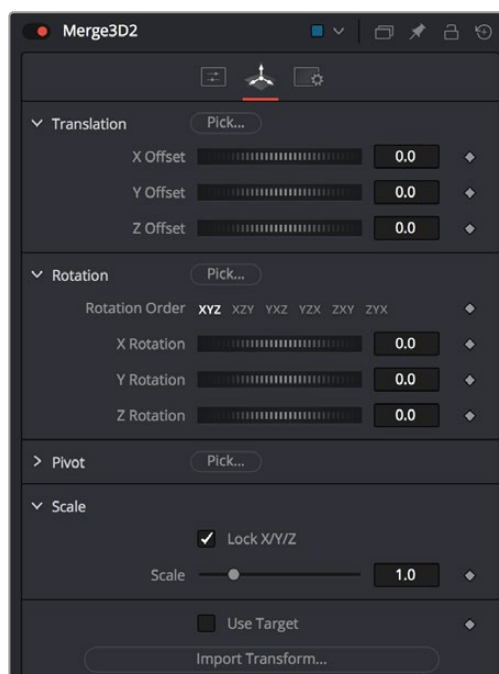
The type of geometry, the renderer, and the state of the lighting can all be set by right-clicking the viewer and choosing options from the contextual menu. Each viewer supports A and B buffers to assist with comparing multiple materials.

Methods of working with the Material Viewer:

- You can change the shape of the previewed geometry by right-clicking the viewer and choosing an option from the Shape submenu of the contextual menu. The geometry that the material is applied to is locked to the center of the viewer and scaled to fit. It is not possible to pan or scale the Material Viewer.
- The Material Viewer can be rotated to provide a different angle on the material by holding Option while pressing the middle mouse button and dragging to the left and right.
- You can adjust the position of the light used to preview the material by dragging with the middle mouse button. Or, you can right-click the viewer and choose an option from the Lighting > Light Position submenu of the contextual menu.
- You can also toggle lighting off and on by right-clicking the viewer and choosing Lighting > Enable Lighting from the contextual menu.
- You can choose the renderer used to preview the material by right-clicking the viewer and choosing an option from the Renderer submenu of the contextual menu.

Transformations

Merge3D, 3D Objects, and Transform3D all have Transform parameters that are collected together into a Transform tab in the Inspector. The parameters found in this tab affect how the object is positioned, rotated, and scaled within the scene.



The Transform tab of a Merge3D node

The Translation parameters are used to position the object in local space, the Rotation parameters affect the object's rotation around its own center, and the Scale slider(s) affect its size (depending on whether or not they're locked together). The same adjustments can be made in the viewer using onscreen controls.

Onscreen Transform Controls

When an object is selected, it displays onscreen Transform controls in the viewers that allow you to adjust the object's position, rotation, and scale. Buttons in the Transform toolbar allow you to switch modes, or you can use the keyboard shortcuts.

To switch Transform modes, use the following keyboard shortcuts:

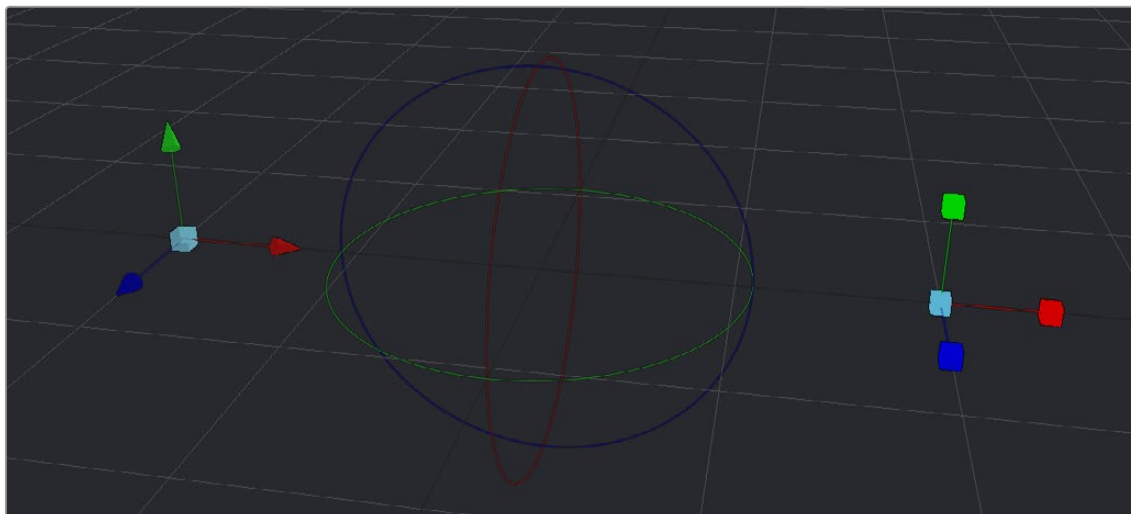
- Press Q for Position
- Press W for Rotation
- Press E for Scaling



The Position, Rotation, and Scale modes in the Transform toolbar

Using Onscreen Transform Controls

In all three modes, red indicates the object's local X-axis, green the Y-axis, and blue the Z-axis, respectively (just remember RGB = XYZ). You can drag directly on the red, green, or blue portion of any onscreen control to constrain the transform to that axis, or if you drag the center of the onscreen control, you can apply a transform without constraints. Holding Option and dragging in the viewer allows you to freely translate in all three axes without clicking on a specific control.



From left to right, the Position, Rotation, and Scale onscreen Transform controls

If the Scale's Lock XYZ checkbox is enabled in the Inspector, only the overall scale of the object is adjusted by dragging the red or center onscreen control, while the green and blue portions of the onscreen control have no effect. If you unlock the parameters, you are able to scale an object along individual axes separately to squish or stretch the object.

Selecting Objects

With the onscreen controls visible in the viewer, you can select any object by clicking on its center control. Alternatively, you can also select any 3D object by clicking its node in the Node Editor.

Pivot

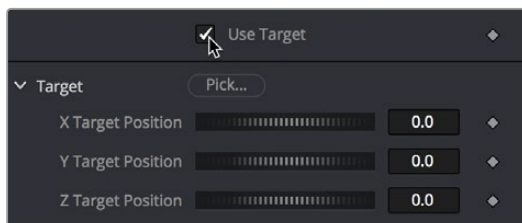
In 3D scenes, objects rotate and scale around an axis called a pivot. By default, this pivot goes through the object's center. If you want to move the pivot so it is offset from the center of the object, you can use the X, Y, and Z Pivot parameters in the Inspector.

Target

Targets are used to help orient a 3D object to a specific point in the scene. No matter where the object moves, it will rotate in the local coordinate system so that it always faces its target, which you can position and animate.

To enable a target for a 3D object:

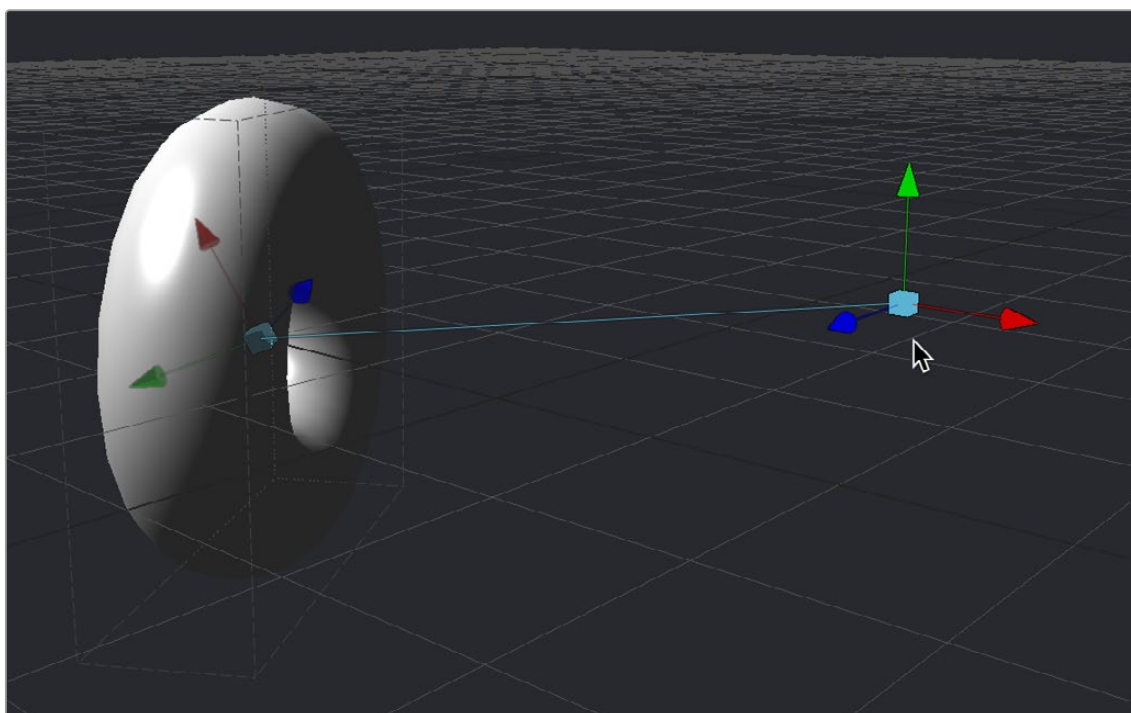
- 1 Select that object's node.
- 2 Open the object's Transform panel in the Inspector.
- 3 Turn on the Use Target checkbox.



Turning on the Use Target checkbox of a 3D object

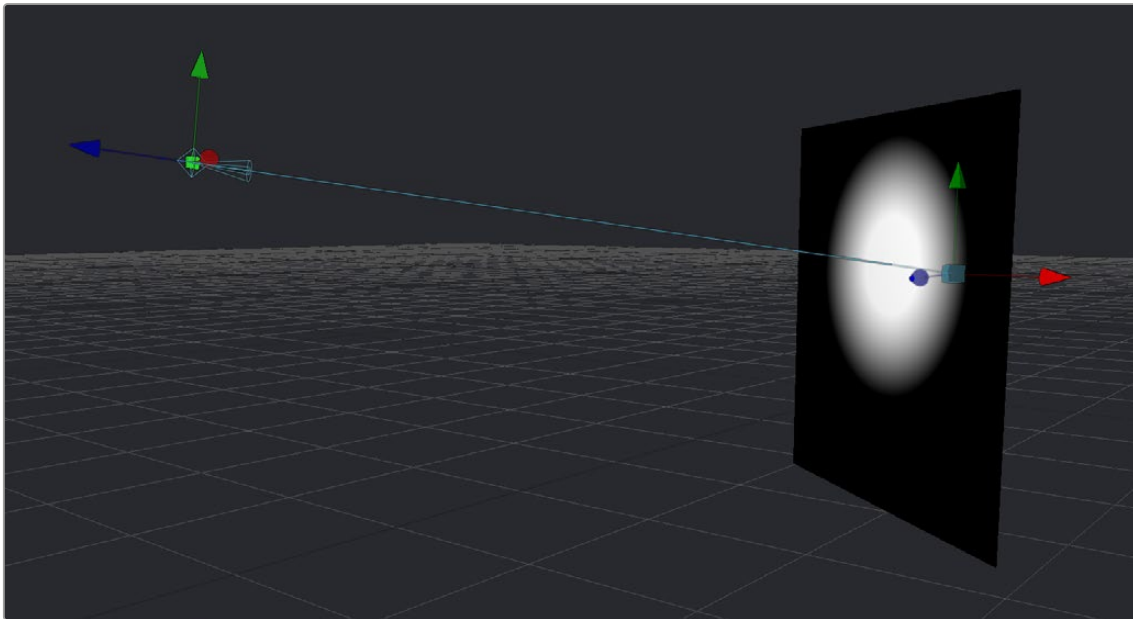
- 4 Use the X/Y/Z Target Position controls in the Inspector or the Target onscreen control in the viewer to position the target and in turn position the object it's attached to.

In the viewer, a line is drawn between the target and the center of the 3D object it's attached to, to show the relationship between these two sets of controls. Whenever you move the target, the object is automatically transformed to face its new position.



A torus facing its onscreen Target controls

For example, if a spotlight is required in the scene to point at an image plane, enable the spotlight's target in the Transform tab and connect the target's XYZ position to the image plane's XYZ position. Now, no matter where the spotlight is moved, it will rotate to face the image plane.

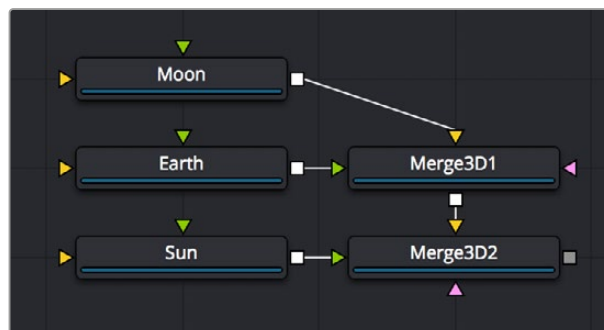


A light made to face the wall using its enabled target control

Parenting

One of the many advantages of the node-based approach to 3D compositing is that parenting between objects becomes implicit in the structure of a 3D node tree. The basis for all parenting is the Merge3D node. If you're careful about how you connect the different 3D objects you create for your scene, you can use multiple Merge3D nodes to control which combinations of objects are transformed and animated together, and which are transformed and animated separately.

For example, picture a scene with two spheres that are both connected to a Merge3D. The Merge3D can be used to rotate one sphere around the other, like the moon around the earth. Then the Merge3D can be connected to another Merge3D to create the earth and the moon orbiting around the sun.



One Merge3D with two spheres parented to another Merge3D and parenting using three connected spheres

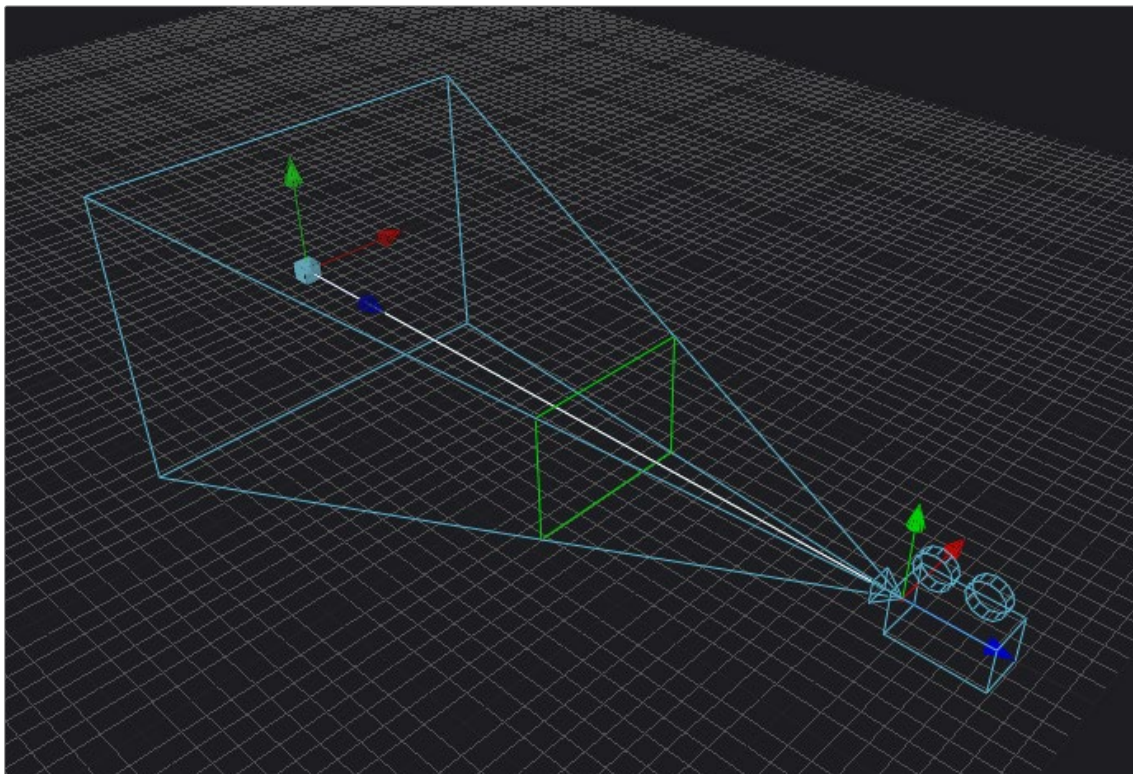
Here are the two simple rules of transforming parented Merge3D nodes:

- Transforms and animation applied to a Merge3D are also applied to all 3D objects connected to that Merge3D node, including cameras, lights, geometry, and other merge nodes connected upstream.
- Transforms and animation applied to upstream merge nodes don't affect downstream merge nodes.

Cameras

When setting up and animating a 3D scene, the metaphor of a camera is one of the most comprehensible ways of framing how you want that scene to be rendered out, as well as animating your way through the scene. Additionally, compositing artists are frequently tasked with matching cameras from live-action clips, or matching cameras from 3D applications.

To accommodate all these tasks, Fusion provides a flexible Camera3D node with common camera controls such as Angle of View, Focal Length, Aperture, and Clipping planes, to either set up your own camera or to import camera data from other applications. The Camera3D node is a virtual camera through which the 3D environment can be viewed.



A camera displayed with onscreen Transform controls in the viewer; the Focal Plane indicator is enabled in green

Cameras are typically connected and viewed via a Merge3D node; however, you can also connect cameras upstream of other 3D objects if you want that camera to transform along with that object when it moves.

Quickly Viewing a Scene Through a Camera

When you've added a camera to a scene, you can quickly view the scene "through the camera" by setting up the following.

To view the scene through the camera:

- 1 Select the Merge3D node that the camera is connected to, or any node downstream of that Merge3D.
- 2 Load the selected Merge3D or downstream node into a viewer.
- 3 Right-click on the axis label in the bottom corner of the viewer and choose the camera name.

The viewer's frame may be different from the camera frame, so it may not match the true boundaries of the image that will be rendered by the Renderer3D node. If there is no Renderer3D node added to your scene yet, you can use Guides that represent the camera's framing.

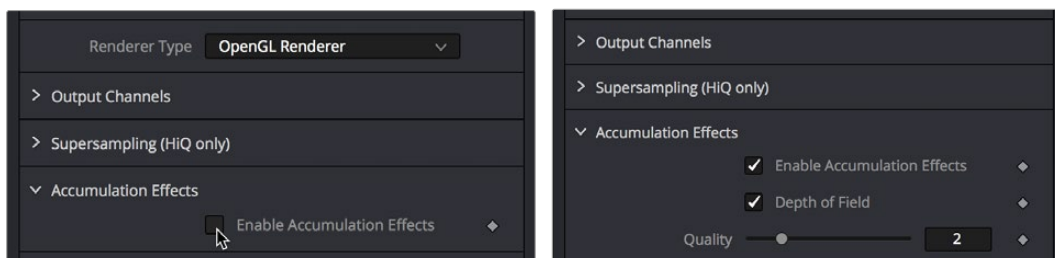
For more information about Guides, see *Chapter 69, "Using Viewers."* in the DaVinci Resolve Reference Manual or Chapter 7 in the Fusion Reference Manual.

Plane of Focus and Depth of Field

Cameras have a plane of focus, for when depth of field rendering is available. Here's the procedure for enabling depth of field rendering in your scenes.

To render depth of field in a 3D scene:

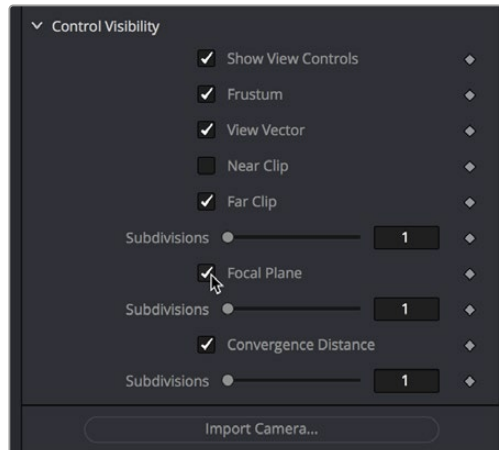
- 1 You must add a Renderer3D node at the end of your 3D scene.
- 2 Select the Renderer3D node, and set the Renderer Type to OpenGL Renderer.
- 3 Open the Accumulation Effects disclosure control that appears, and turn on the Enable Accumulation Effects checkbox in the OpenGL render.



Turning on Enable Accumulation Effects enables additional depth of field controls

Turning on "Enable Accumulation Effects" exposes a Depth of Field checkbox along with Quality and Amount of DoF Blur sliders that let you adjust the depth of field effect. These controls affect only the perceived quality of the depth of field that is rendered. The actual depth of field that's generated depends solely on the setup of the camera and its position relative to the other 3D objects in your scene.

When you select your scene's Camera3D node to view its controls in the Inspector, a new Focal Plane checkbox appears in the Control Visibility group. Turning this on lets you see the green focal plane indicator in the 3D Viewer that lets you visualize the effect of the Focal Plane slider, which is located in the top group of parameters in the Camera3D node's Controls tab.



Turning on the Focal Plane checkbox in the Camera3D node

For more information about these specific camera controls, see *Chapter 89, “3D Nodes,”* in the DaVinci Resolve Reference Manual or Chapter 29 in the Fusion Reference Manual.

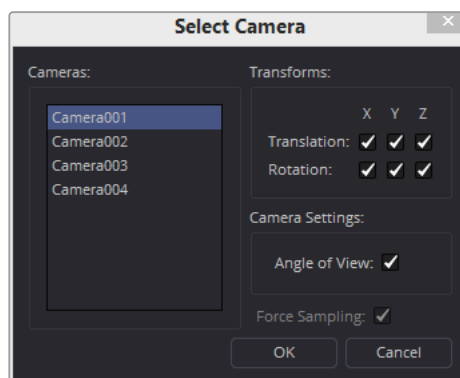
Importing Cameras

If you want to match cameras between applications, you can import camera paths and positions from a variety of popular 3D applications. Fusion is able to import animation splines from Maya and XSI directly with their own native spline formats. Animation applied to cameras from 3ds Max and LightWave are sampled and keyframed on each frame.

To import a camera from another application, do the following:

- 1 Select the camera in the Node Editor.
- 2 At the bottom of the Inspector, click the Import Camera button.
- 3 In the file browser, navigate to and select the scene that contains the camera you want to import.

A dialog box with several options will appear. When the Force Sampling checkbox is enabled, Fusion will sample each frame of the motion, regardless of the format.



The Select Camera dialog

TIP: When importing parented or rigged cameras, baking the camera animation in the 3D application before importing it into Fusion often produces more reliable results.

Lighting and Shadows

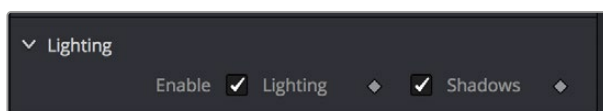
You can add light sources to a scene to create very detailed lighting environments and atmosphere. There are four different types of lights you can use in 3D scenes: ambient, directional, point, and spotlights.

Enabling Lighting in the Viewer

A scene without lights uses a default directional light, but this automatically disappears once you add a 3D light object. However, even when you add light objects to your scene, lighting and shadows won't be visible in the viewer unless you first enable lighting in the viewer contextual menu by right-clicking anywhere within a viewer and choosing 3D Options > Lighting or Shadows to turn on one or both.

Enabling Lighting to Be Rendered

Lighting effects won't be rendered in the Render3D node until the Enable Lighting and/or Shadows checkboxes are checked in the Inspector.



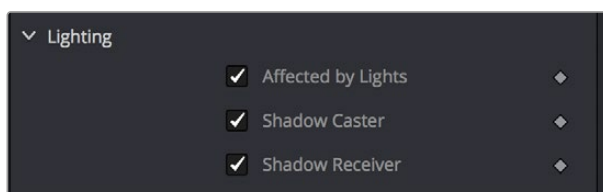
The Lighting button under the viewer

NOTE: When lighting is disabled in either the viewer or final renders, the image will appear to be lit by a 100% ambient light.

Controlling Lighting within Each 3D Object

All nodes that create or merge geometry also include lighting options that are used to choose how each object is affected by light:

- Merge3D nodes have a Pass Through Lights checkbox that determines whether lights attached to an upstream Merge3D node also illuminate objects attached to downstream Merge3D nodes.
- ImagePlane3D, Cube3D, Shape3D, Text3D, and FBXMesh3D nodes have a set of Lighting controls that let you turn three controls on and off: Affected by Lights, Shadow Caster, and Shadow Receiver.



3D objects have individual lighting controls that let you control how each object interacts with light and shadows

Lighting Types Explained

Here's a more detailed explanation of each type of light in Fusion.

Ambient Light

You use ambient light to set a base light level for the scene, since it produces a general uniform illumination of the scene. Ambient light exists everywhere without appearing to come from any particular source; it cannot cast shadows and will tend to fill in shadowed areas of a scene.

Directional Light

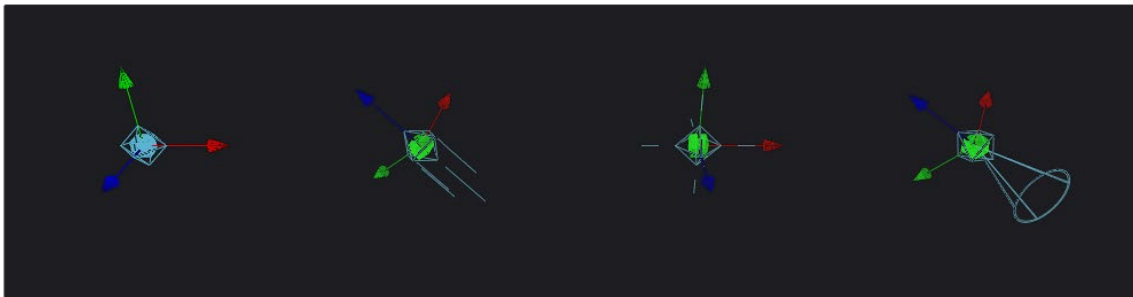
A directional light is composed of parallel rays that light up the entire scene from one direction, creating a wall of light. The sun is an excellent example of a directional light source.

Point Light

A point light is a well defined light that has a small clear source, like a light bulb, and shines from that point in all directions.

Spotlight

A spotlight is an advanced point light that produces a well defined cone of light with falloff. This is the only light that produces shadows.

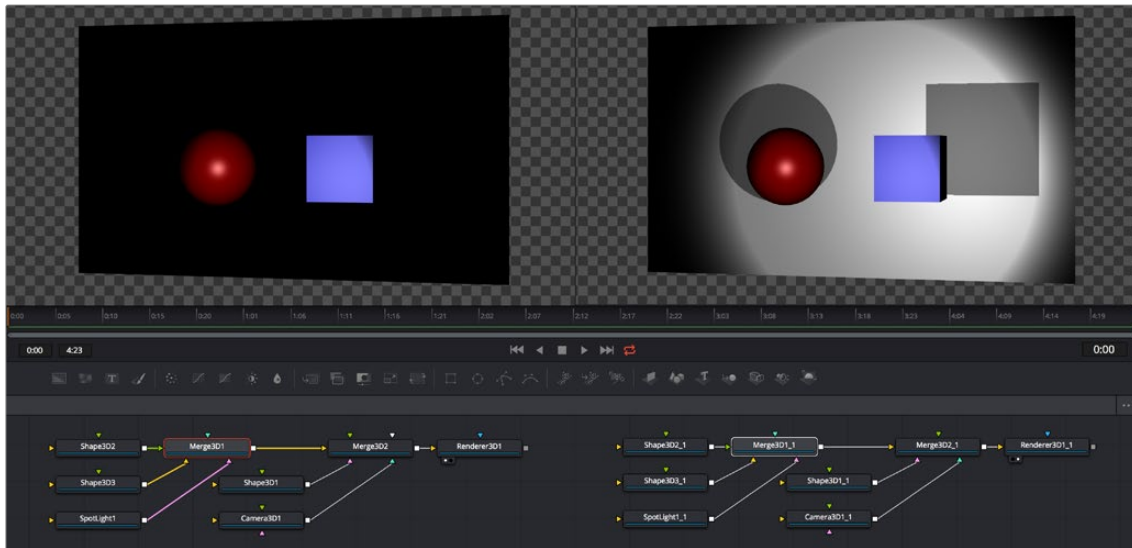


From left to right: Directional light, point light, and spotlight

All of the Light nodes display onscreen controls in the viewer, although not all controls affect every light type. In the case of the ambient light, the position has no effect on the results. The directional light can be rotated, but position and scale will be ignored. The point light ignores rotation. Both position and rotation apply to the spotlight.

Lighting Hierarchies

Lights normally do not pass through a Merge, since the Pass Through Lights checkbox is off by default. This provides a mechanism for controlling which objects are lit by which lights. For example, in the following two node trees, two shapes and an ambient light are combined with a Merge3D node, which is then connected to another Merge3D node that's also connected to a plane and a spotlight. At the left, the first Merge3D node of this tree has Pass Through Lights disabled, so you can only see the two shapes lit. At the right, Pass Through Lights has been enabled, so both the foreground shapes and the background image plane receive lighting.

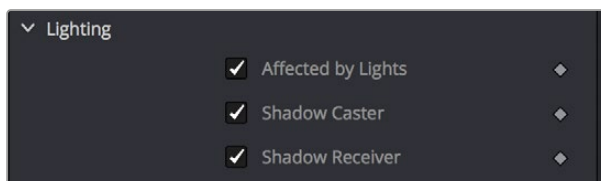


Pass Through Lights is disabled, so only the front two shapes are illuminated (left) Pass Through Lights is enabled, so all shapes connected to both Merge3D nodes are illuminated (right)

Lighting Options

Most nodes that generate geometry have additional options for lighting. These options are used to determine how each individual object reacts to lights and shadows in the scene.

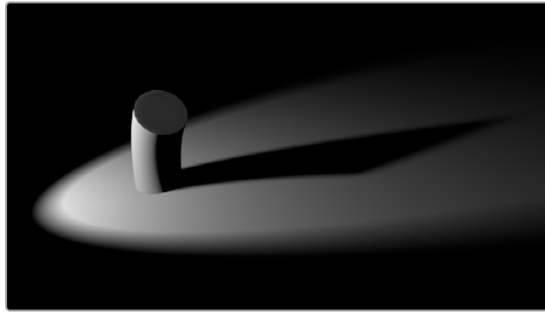
- **Affected By Lights:** If the Affected By Lights checkbox is enabled, lights in the scene will affect the geometry.
- **Shadow Caster:** When enabled, the object will cast shadows on other objects in the scene.
- **Shadow Receiver:** If this checkbox is enabled, the object will receive shadows.



3D objects have individual lighting controls that let you control how each object interacts with light and shadows

Shadows

Spotlight, Point Light, and Directional light can all cast shadows in the 3D viewer, however Point and Directional lights need to have Hardware Renderer set as the Rendering Type in the Renderer3D node. Spotlight can work with either the Hardware or Software renderers. These lights cast shadows by default, although these shadows will not be visible in the viewer until shadows are enabled using the viewer toolbar button. Shadows will not appear in the output of the Renderer3D unless the Shadows option is enabled for that renderer. If you want to prevent a spotlight from casting shadows, you can disable the Enable Shadows checkbox in the node's Inspector.



An image with spotlight casting a variable soft shadow

For more information on shadow controls, see the “Spotlight” section of *Chapter 90, “3D Light Nodes,”* in the DaVinci Resolve Reference Manual or Chapter 30 in the Fusion Reference Manual.

Shadow Maps

A shadow map is an internal depth map that specifies each pixel's depth in the scene. This information is used to assemble the shadow layer created from a spotlight. All the controls for the shadow map are found in the Spotlight Inspector.

The quality of the shadow produced depends greatly on the size of the shadow map. Larger maps generate better-looking shadows but will take longer to render. The wider the cone of the spotlight, or the more falloff in the cone, the larger the shadow map will need to be to produce useful quality results. Setting the value of the Shadow Map Size control sets the size of the depth map in pixels.

Generally, through trial and error, you'll find a point of diminishing returns where increasing the size of the shadow map no longer improves the quality of the shadow. It is not recommended to set the size of the shadow maps any larger than they need to be.

The Shadow Map Proxy control is used to set a percentage by which the shadow map is scaled for fast interactive previews, such as Autoproxy and LoQ renders. A value of 4, for example, represents a 40% proxy.

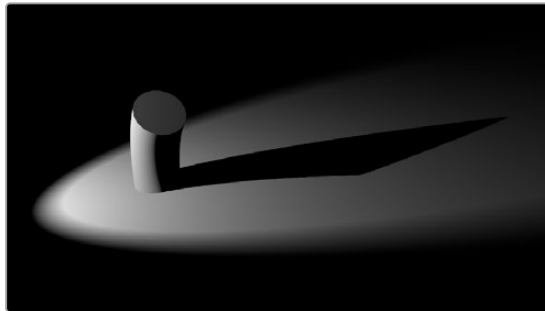
Shadow Softness

By default, the spotlight generates shadows without soft edges, but there are options for constant and variable soft shadows. Hard-edged shadows will render significantly faster than either of the Soft Shadow options. Shadows without softness will generally appear aliased, unless the shadow map size is large enough. In many cases, softness is used to hide the aliasing rather than increasing the shadow map to preserve memory and avoid exceeding the graphics hardware capabilities.



Soft Shadow controls in the Control panel

Setting the spotlight's shadow softness to None will render crisp and well-defined shadows. The Constant option will generate shadows where the softness is uniform across the shadow, regardless of the shadow's distance from the casting geometry. The Variable option generates shadows that become softer as they get farther from the geometry that is casting the shadow. This is a more realistic effect, but the shadows are somewhat harder to control. When this option is selected, additional controls for adjusting the falloff of the shadow will appear, as well as sliders for the minimum and maximum softness.



Hard shadow cast by a spotlight

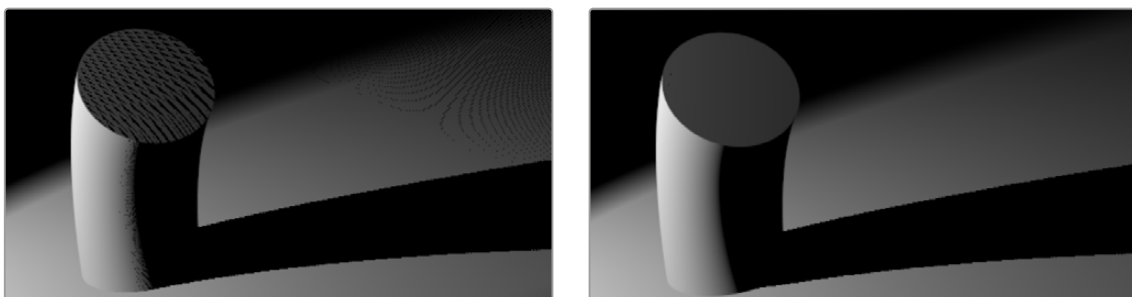
Selecting the Variable option reveals the Spread, Min Softness, and Filter Size sliders. A side effect of the method used to produce variable softness shadows is that the size of the blur applied to the shadow map can become effectively infinite as the shadow's distance from the geometry increases. These controls are used to limit the shadow map by clipping the softness calculation to a reasonable limit.

The filter size determines where this limit is applied. Increasing the filter size increases the maximum possible softness of the shadow. Making this smaller can reduce render times but may also limit the softness of the shadow or potentially even clip it. The value is a percentage of the shadow map size.

For more information, see "Spotlight" in *Chapter 90, "3D Light Nodes,"* in the DaVinci Resolve Reference Manual or Chapter 30 in the Fusion Reference Manual.

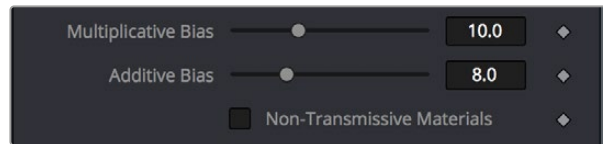
Multiplicative and Additive Bias

Shadows are essentially textures applied to objects in the scene that occasionally result in "fighting." Z-fighting results when portions of an object that should be receiving shadows instead render over the top of the shadow because they effectively exist in the same exact location in 3D space.



Results of shadow map Z-fighting (Left), and the corrected shadow shown using Biasing (Right)

Two Biasing sliders in the Shadows group of Spotlight parameters work by adding a small depth offset to move the shadow away from the surface it is shadowing, eliminating the Z-fighting. When too little bias is added, the objects can self shadow themselves. When too much is added, the shadow can become separated from the surface.



The Multiplicative and Additive Bias sliders, and the Non-Transmissive Materials checkbox, all in the Spotlight Inspector controls

The goal is to adjust the Multiplicative Bias slider until the majority of the Z-fighting is resolved, and then adjust the Additive Bias slider to eliminate the rest. The softer the shadow, the higher the bias will probably have to be. You may even need to animate the bias to get a proper result for some particularly troublesome frames.

Force All Materials Non-Transmissive

How light passes through a semi-transparent material plays an important role in determining the appearance of the shadow an object casts. Normally, this transmittance behavior is defined in each object's Materials tab. However, selecting Force All Materials Non-Transmissive in the Spotlight Inspector overrides this, causing the shadow map produced by the node to ignore transmittance entirely.

Materials and Textures

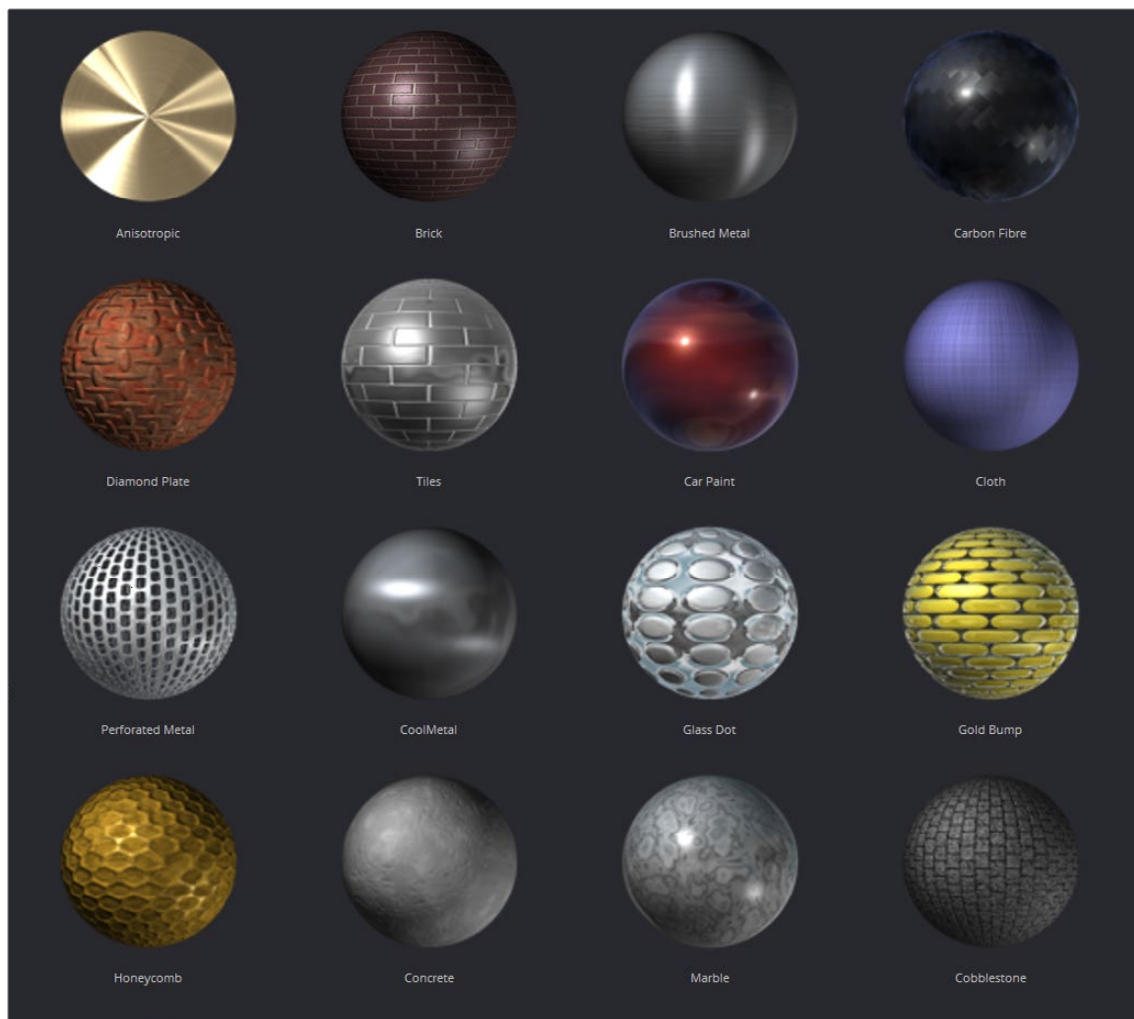
To render a 3D scene, the renderer must take into account the shape of the object as well as its appearance. The geometry of an object defines the shape of the object, while the material applied to the object defines its appearance. Fusion provides a range of options for applying materials and textures to geometry, so you can give your 3D objects the surface qualities you want.

Nodes that describe the geometry's response to light are called illumination models. Blinn, Cook-Torrance, Ward, and Phong are the included illumination models. These nodes are found in the 3D > Material category of nodes in the Effects Library.

Most materials also accept textures, which are typically 2D images. Textures are used to refine the look of an object further, by adding photorealistic details, transparency, or special effects. More complex textures like bump maps, 3D textures, and reflection maps are also available in the 3D > Texture category.

Materials can also be combined to produce elaborate and highly detailed composite materials.

Each node that creates or loads geometry into a 3D scene also assigns a default material. The default material is the Blinn illumination model, but you can override this material using one of several nodes that output a 3D material. Some of these materials provide a greater degree of control over how the geometry reacts to light, providing inputs for diffuse and specular texture maps, bump mapping, and environmental maps, which mimic reflection and refraction.



Material examples from the bin

Material Components

All the standard illumination models share certain characteristics that must be understood.

Diffuse

The Diffuse parameters of a material control the appearance of an object where light is absorbed or scattered. This diffuse color and texture are the base appearance of an object, before taking into account reflections. The opacity of an object is generally set in the diffuse component of the material.

Alpha

The Alpha parameter defines how much the object is transparent to diffuse light. It does not affect specular levels or color. However, if the value of alpha, either from the slider or a Material input from the diffuse color, is very close to or at zero, those pixels, including the specular highlights, will be skipped and disappear.

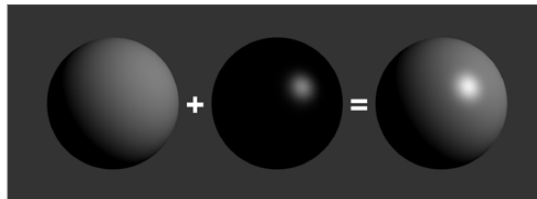
Opacity

The Opacity parameter fades out the entire material, including the specular highlights. This value cannot be mapped; it is applied to the entire material.

Specular

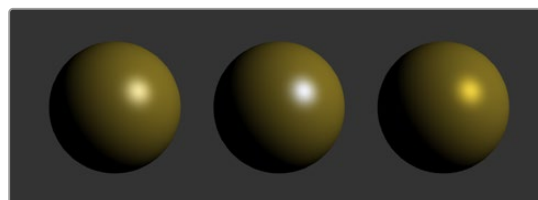
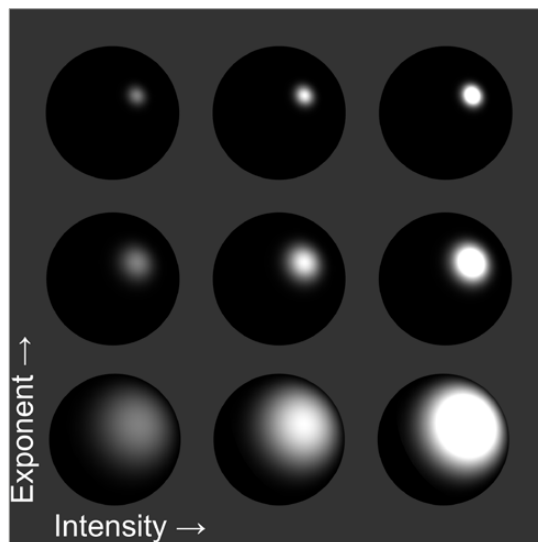
The Specular parameters of a material control the highlight of an object where the light is reflected to the current viewpoint. This causes a highlight that is added to the diffuse component. The more specular a material is, the glossier it appears. Surfaces like plastics and glass tend to have white specular highlights, whereas metallic surfaces like gold have specular highlights that tend to inherit their color from the material color.

Specularity is made up of color, intensity, and exponent. The specular color determines the color of light that reflects from a shiny surface. Specular intensity is how bright the highlight will be.



Three spheres, left to right: diffuse only, specular only, and combined

The specular exponent controls the falloff of the specular highlight. The larger the value, the sharper the falloff and the smaller the specular component will be.



Left to right: white, complimentary, and matching specular colors

Transmittance

When using the software renderer, the Transmittance parameters control how light passes through a semi-transparent material. For example, a solid blue pitcher will cast a black shadow, but one made of translucent blue plastic would cast a much lower density blue shadow. The transmittance parameters are essential to creating the appearance of stained glass.

TIP: You can adjust the opacity and transmittance of a material separately. It is possible to have a surface that is fully opaque yet transmits 100% of the light arriving upon it, so in a sense it is actually a luminous/emissive surface.

Transmissive surfaces can be further limited using the Alpha and Color Detail control.

Attenuation

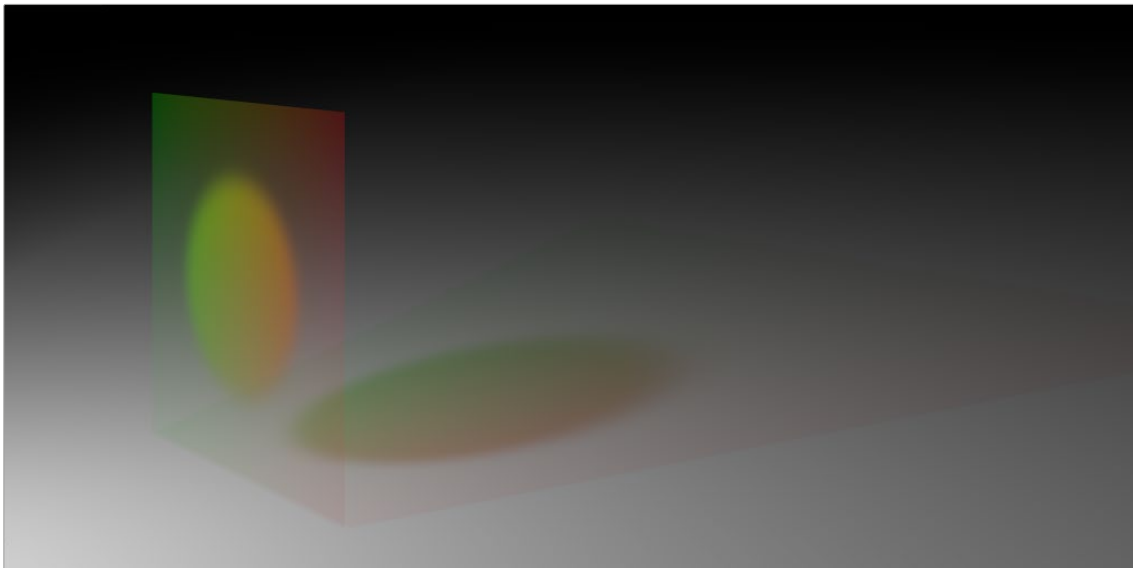
The transmittance color determines how much color is passed through the object. For an object to have fully transmissive shadows, the transmittance color must be set to $RGB = (1, 1, 1)$, which means 100% of green, blue, and red light passes through the object. Setting this color to $RGB = (1, 0, 0)$ means that the material will transmit 100% of the red arriving at the surface but none of the green or blue light.

Alpha Detail

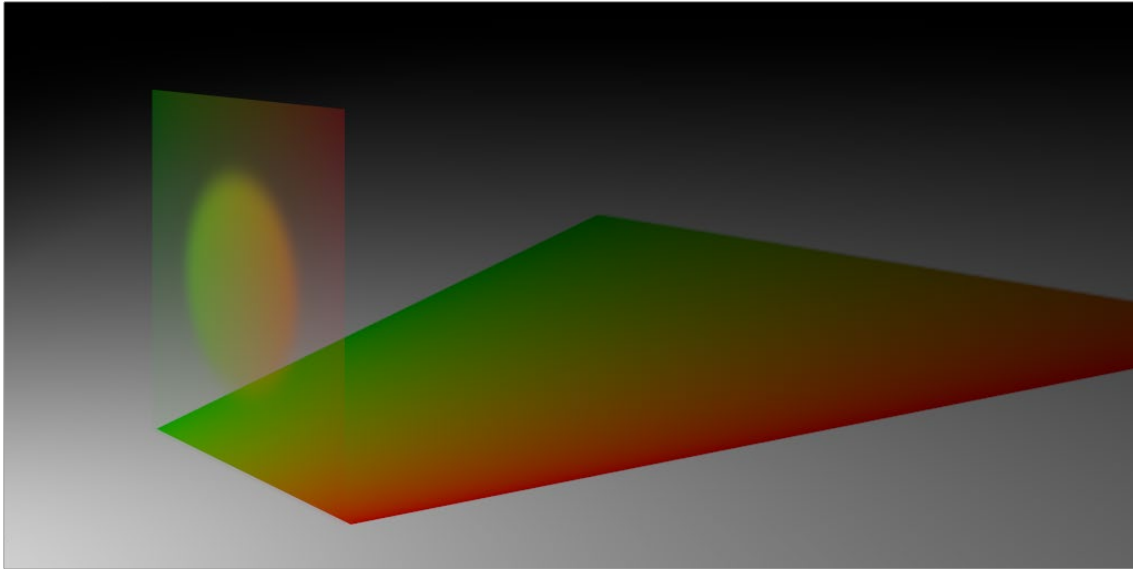
When the Alpha Detail slider is set to 0, the non-zero portions of the alpha channel of the diffuse color are ignored and the opaque portions of the object casts a shadow. If it is set to 1, the alpha channel determines how dense the object casts a shadow.

NOTE: The OpenGL renderer ignores alpha channels for shadow rendering, resulting in a shadow always being cast from the entire object. Only the software renderer supports alpha in the shadow maps.

The following examples for Alpha Detail and Color Detail cast a shadow using this image. It is a green-red gradient from left to right. The outside edges are transparent, and inside is a small semi-transparent circle.



Alpha Detail set to 1; the alpha channel determines the density of the shadow

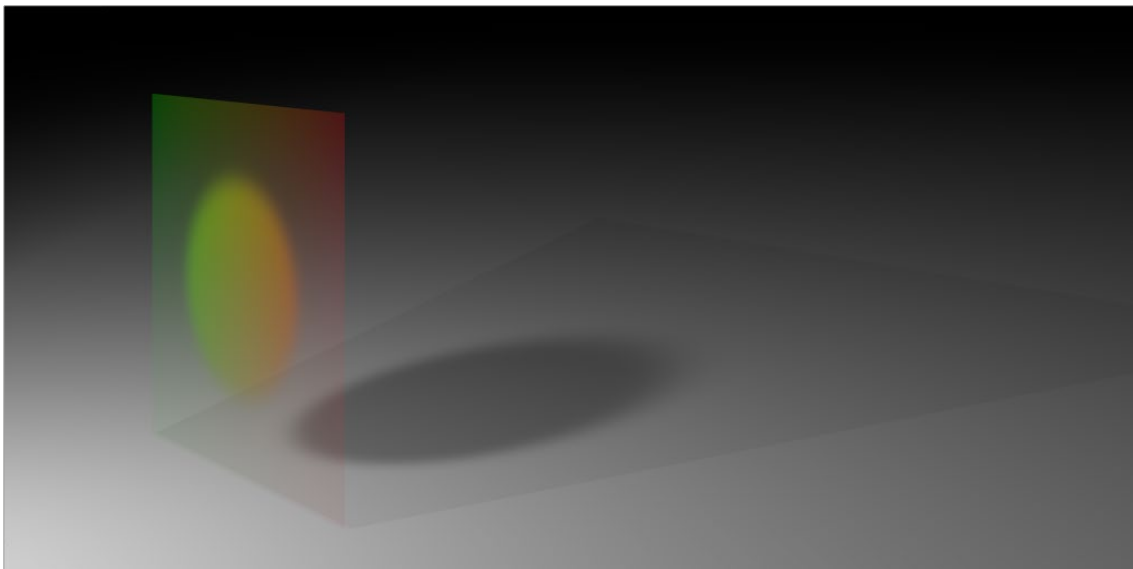


Alpha Detail set to 0; a dense-colored shadow results

Color Detail

Color Detail is used to color the shadow with the object's diffuse color. Increasing the Color Detail slider from 0 to 1 brings in more diffuse color and texture into the shadow.

TIP: The OpenGL renderer will always cast a black shadow from the entire object, ignoring the color. Only the software renderer supports color in the shadow maps.



Color Detail set to 0; no color is visible in the shadow.

Saturation

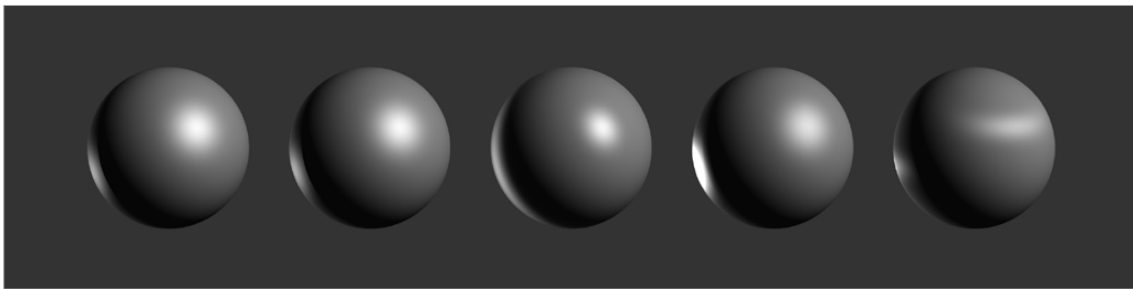
Saturation will allow the diffuse color texture to be used to define the density of the shadow without affecting the color. This slider lets you blend between the full color and luminance only.

Transmittance and Shadows

The transmittance of an object's material plays an important role in determining the appearance of the shadow it casts. Normally, the transmittance behavior is defined in each object's Materials tab as explained above. However, selecting Force All Materials Non-Transmissive in the Spotlight Inspector overrides this, causing the shadow map produced by the spotlight to ignore transmittance entirely.

Illumination Models

Now that you understand the different components that make up a material or shader, we'll look at them more specifically. Illumination models are advanced materials for creating realistic surfaces like plastic, wood, or metal. Each illumination model has advantages and disadvantages, which make it appropriate for particular looks. An illumination model determines how a surface reacts to light, so these nodes require at least one light source to affect the appearance of the object. Four different illumination models can be found in the Nodes > 3D > Material menu.



Illumination models left to right: Standard, Blinn, Phong, Cook-Torrance, and Ward

Standard

The Standard material provides a default Blinn material with basic control over the diffuse, specular, and transmittance components. It only accepts a single texture map for the diffuse component with the alpha used for opacity. The Standard Material controls are found in the Material tab of all nodes that load or create geometry. Connecting any node that outputs a material to that node's Material Input will override the Standard material, and the controls in the Material tab will be hidden.

Blinn

The Blinn material is a general purpose material that is flexible enough to represent both metallic and dielectric surfaces. It uses the same illumination model as the Standard material, but the Blinn material allows for a greater degree of control by providing additional texture inputs for the specular color, intensity, and exponent (falloff), as well as bump map textures.

Phong

The Phong material produces the same diffuse result as Blinn, but with wider specular highlights at grazing incidence. Phong is also able to make sharper specular highlights at high exponent levels.

Cook-Torrance

The Cook-Torrance material combines the diffuse illumination model of the Blinn material with a combined microfacet and Fresnel specular model. The microfacets need not be present in the mesh or bump map; they are represented by a statistical function, Roughness, which can be mapped. The Fresnel factor attenuates the specular highlight according to the Refractive Index, which can be mapped.

Ward

The Ward material shares the same diffuse model as the others but adds anisotropic highlights, ideal for simulating brushed metal or woven surfaces, as the highlight can be elongated in the U or V directions of the mapping coordinates. Both the U and V spread functions are mappable.

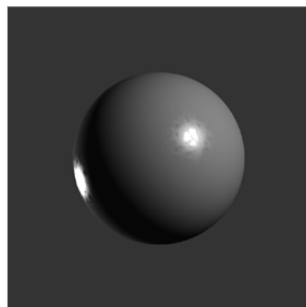
This material does require properly structured UV coordinates on the meshes it is applied to.

Textures

Texture maps modify the appearance of a material on a per-pixel basis. This is done by connecting an image or other material to the inputs on the Material nodes in the Node Editor. When a 2D image is used, the UV mapping coordinates of the geometry are used to fit the image to the geometry, and when each pixel of the 3D scene is rendered, the material will modify the material input according to the value of the corresponding pixel in the map.

TIP: UV Mapping is the method used to wrap a 2D image texture onto 3D geometry. Similar to X and Y coordinates in a frame, U and V are the coordinates for textures on 3D objects.

Texture maps are used to modify various material inputs, such as diffuse color, specular color, specular exponent, specular intensity, bump map, and others. The most common uses of texture maps is the diffuse color/opacity component.



The Fast Noise texture used to control the roughness of a Cook-Torrance material

A node that outputs a material is frequently used, instead of an image, to provide other shading options. Materials passed between nodes are RGBA samples; they contain no other information about the shading or textures that produced them.

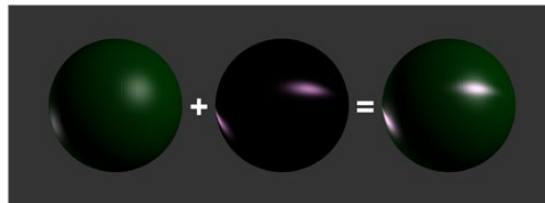


The Texture2D node is used to translate a texture in the UV space of the object, as well as set the filtering and wrap mode

Composite Materials

Building complex materials is as easy as connecting the output of a Material node to one of the Material inputs of another Material or Texture node. When a Material input is supplied just as with a 2D image, its RGBA values are used per pixel as a texture. This allows for very direct compositing of shaders.

For instance, if you want to combine an anisotropic highlight with a Blinn material, you can take the output of the Blinn, including its specular, and use it as the diffuse color of the Ward material. Or, if you do not want the output of the Blinn to be relit by the Ward material, you can use the Channel Boolean material to add the Ward material's anisotropic specular component to the Blinn material with a greater degree of control.



Combining an anisotropic highlight with a Blinn material using the Channel Boolean material

Reflections and Refractions

Environment maps can be applied with the Reflect material in the 3D > Material category. This node can be used to simulate reflections and refractions on an object. Reflections are direct-bounce light that hits an object, while refractions simulate the distortion of light seen through semi-translucent surfaces.

The reflections and refractions use an environment mapping technique to produce an approximation that balances realistic results with greater rendering performance. Environment maps assume an object's environment is infinitely distant from the object and rendered into a cubic or spherical texture surrounding the object.

The Nodes > 3D > Texture > Cube Map and Sphere Map nodes can be used to help create environment maps, applying special processing and transforms to create the cubic or spherical coordinates needed.



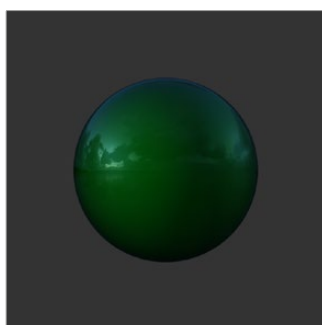
Sphere map example

To produce reflections with real-time interactive feedback at a quality level appropriate for production environment maps, you make some trade-offs on functionality when compared with slower but physically accurate raytraced rendering. Environment-mapped reflections and refractions do not provide self-reflection or any other kind of interaction between different objects. In particular, this infinite distance assumption means that objects cannot interact with themselves (e.g., the reflections on the handle of a teapot will not show the body of the teapot). It also means that objects using the same cube map will not inter-reflect with each other. For example, two neighboring objects would not reflect each other. A separate cube map must be rendered for each object.

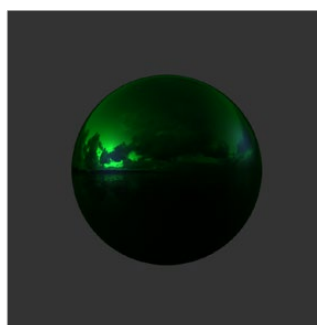
The Reflect node outputs a material that can be applied to an object directly, but the material does not contain an illumination model. As a result, objects textured directly by the Reflect node will not respond to lights in the scene. For this reason, the Reflect node is usually combined with the Blinn, Cook-Torrance, Phong, or Ward nodes.

Reflection

Reflection outputs a material making it possible to apply the reflection or refraction to other materials either before or after the lighting model with different effects.



A Blinn material connected to a background material input of the Reflect. This causes the reflection to be added to the Blinn output.



A Reflect is connected to the Diffuse Color component of the Blinn, causing the reflection to be multiplied by the diffuse color and modulated by the lighting.

Refraction

Refraction occurs only where there is transparency in the background material, which is generally controlled through the Opacity slider and/or the alpha channel of any material or texture used for the Background Material Texture input. The Reflect node provides the following material inputs:

- **Background Material:** Defines both the opacity for refraction and the base color for reflection.
- **Reflection Color Material:** The environment reflection.
- **Reflection Intensity Material:** A multiplier for the reflection.
- **Refraction Tint Material:** The environment refraction.
- **Bump Map Texture:** Normal perturbing map for environment reflection/refraction vectors.

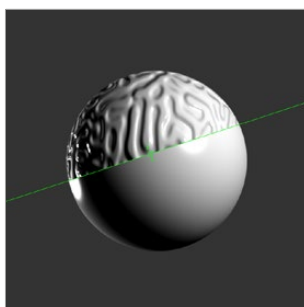
Working with reflection and refraction can be tricky. Here are some techniques to make it easier:

- Typically, use a small amount of reflection, between 0.1 and 0.3 strength. Higher values are used for surfaces like chrome.
- Bump maps can add detail to the reflections/refractions. Use the same bump map in the Illumination model shader that you combine with Reflect.

- When detailed reflections are not required, use a relatively small cube map, such as 128 x 128 pixels, and blur out the image.
- The alpha of refracted pixels is set to 1 even though the pixels are technically transparent. Refracted pixels increase their alpha by the reflection intensity.
- If the refraction is not visible even when a texture is connected to the Refraction Tint Material input, double-check the alpha/opacity values of the background material.

Bump Maps

Bump mapping helps add details and small irregularities to the surface appearance of an object. Bump mapping modifies the geometry of the object or changes its silhouette.



Split screen of a sphere—half with bump map, half without

To apply a bump map, you typically connect an image containing the bump information to the BumpMap node. The bump map is then connected to the Bump input of a Material node. There are two ways to create a bump map for a 3D material: a height map and a bump map.

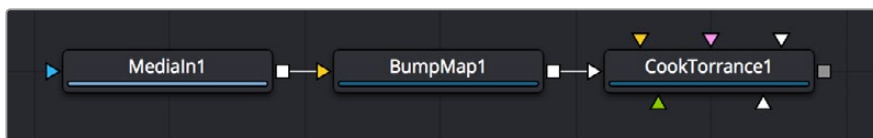


Image connected to a BumpMap connected to a CookTorrance material node

Using a Height Map

A height map is an image where the value of a pixel represents the height. It is possible to select which color channel is used for bump creation. White means high and black means low; however, it is not the value of a pixel in the height map that determines the bumpiness, but rather how the value changes in the neighborhood of a pixel.

Using a Bump Map

A bump map is an image containing normals stored in the RGB channels.

TIP: Normals are generated by 3D modeling and animation software as a way to trick the eye into seeing smooth surfaces, even though the geometry used to create the models uses only triangles to build the objects.

Normals are 3 float values (nx, ny, nz) whose components are in the range $[-1, +1]$. Because you can store only positive values in Fusion's integer images, the normals are packed from the range $[-1, +1]$

to the range [0, 1] by multiplying by 0.5 and adding 0.5. You can use Brightness Contrast or a Custom node to do the unpacking.

If you were to connect a bump map directly to the bump map input of a material, it will result in incorrect lighting. Fusion prevents you from doing this, however, because Fusion uses a different coordinate system for doing the lighting calculation. You first must use a BumpMap that expects a packed bump map or height map and will do the conversion of the bump map to work correctly.

If your bump mapping doesn't appear correct, here are a few things to look for:

- Make sure you have the nodes connected correctly. The height/bump map should connect into a BumpMap and then, in turn, should connect into the bump map input on a material.
- Change the precision of the height map to get less banding in the normals. For low frequency images, float32 may be needed.
- Adjust the Height scale on the BumpMap. This scales the overall effect of the bump map.
- Make sure you set the type to HeightMap or BumpMap to match the image input. Fusion cannot detect which type of image you have.
- Check to ensure High Quality is on (right-click in the transport controls bar and choose High Quality from the contextual menu). Some nodes like Text+ produce an anti-aliased version in High Quality mode that will substantially improve bump map quality.
- If you are using an imported normal map image, make sure it is packed [0–1] in RGB and that it is in tangent space. The packing can be done in Fusion, but the conversion to tangent space cannot.

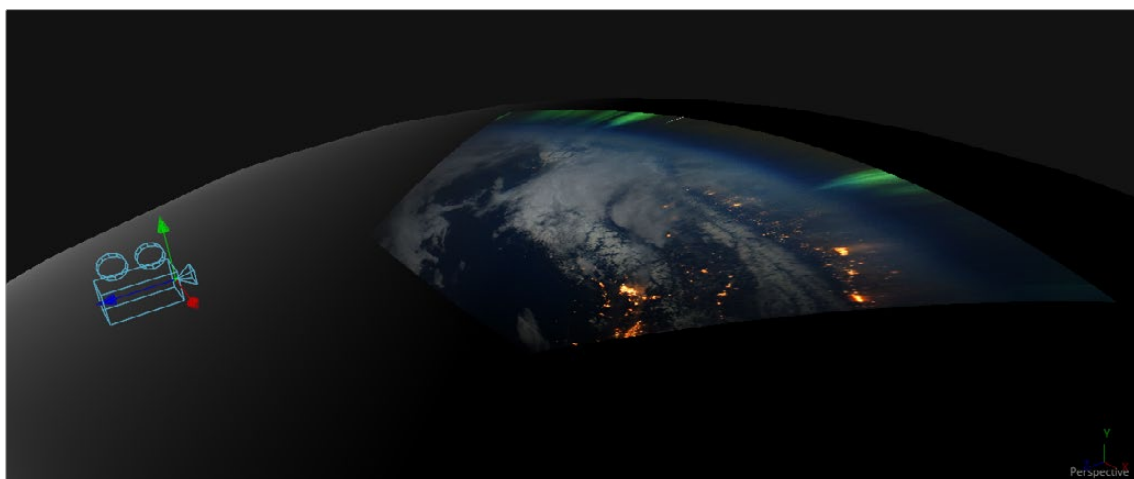
Projection Mapping

Projection is a technique for texturing objects using a camera or projector node. This can be useful for texturing objects with multiple layers, applying a texture across multiple separate objects, projecting background shots from the camera's viewpoint, image-based rendering techniques, and much more.

There are three ways to do projection mapping in Fusion.

Using the Projector/Camera Tool to Project Light

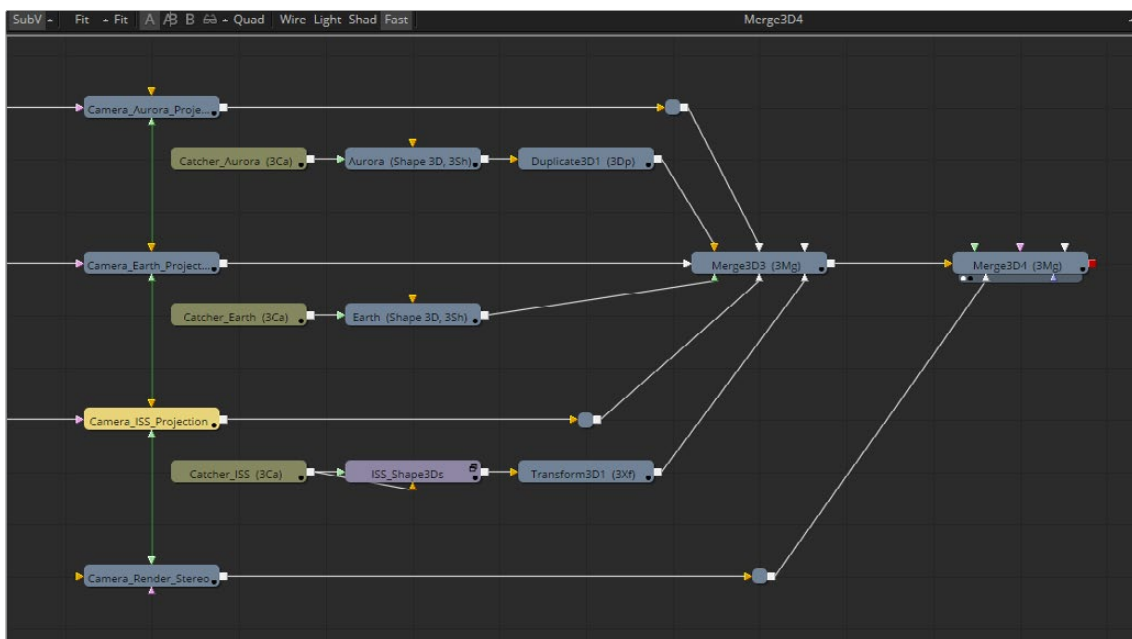
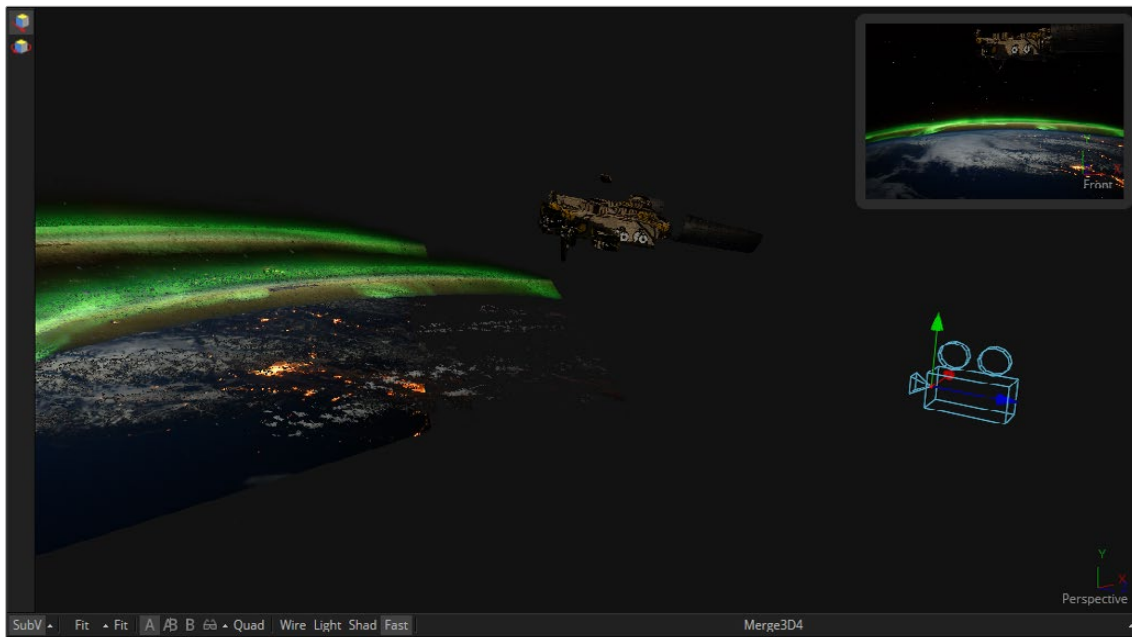
When lighting is enabled, a Camera 3D or Projector 3D can act as a light with all the lighting features. When Camera Projection is enabled or you use a projector, you can choose whether the projection behaves like a spotlight or an ambient light; however, alpha channels cannot be projected. Overlapping projections add together like any other light node. An internal clipping plane (at around 0.01 distance from camera) limits how close the projector or camera can get to the receivers of the projection.



Camera node used for a projection map

Project a Texture onto a Catcher Material

If you do not want to work with light sources, you can use the projector or camera as a texture projector. To work without lighting, a catcher is required in order to receive the texture and apply it to a material. Only objects using this material will receive the projection. This offers some advantages, like the projection of alpha channels, and texturing other channels like specular color or roughness. If the software renderer is used, overlapping projections can be combined in various ways (mean, median, blend, and so on) via the Catcher node. When using the OpenGL renderer, one catcher per projector is used, and the results can be combined using another material. Similar to the Light Projection technique, an internal clipping plane (at around 0.01 distance from camera) limits how close the projector/camera can get to the projection receivers.



Camera projection used with a Catcher node (example from an older version of Fusion)

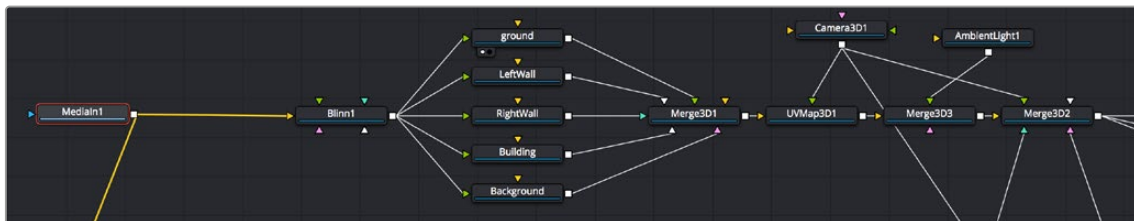
Project Using the UVMap Node

This mode requires a camera and a UVMap3D node downstream of the objects to which the texture is being projected. In the Inspector, when the UVMap Map mode is set to Camera, it gathers the information from the camera and creates new UVs for the input objects, which are used for texturing. Because the UVs are stored in the vertices of the mesh, the object must be tessellated sufficiently.

Textures are assigned to the object like any other texturing technique. The UVs can be locked to the vertices at a chosen frame using the Ref Time slider. This locking only works as long as vertices are not created, destroyed, or reordered (e.g., projection locking will not work on particles because they get created/destroyed, nor will they work on a Cube3D when its subdivision level slider is animated).

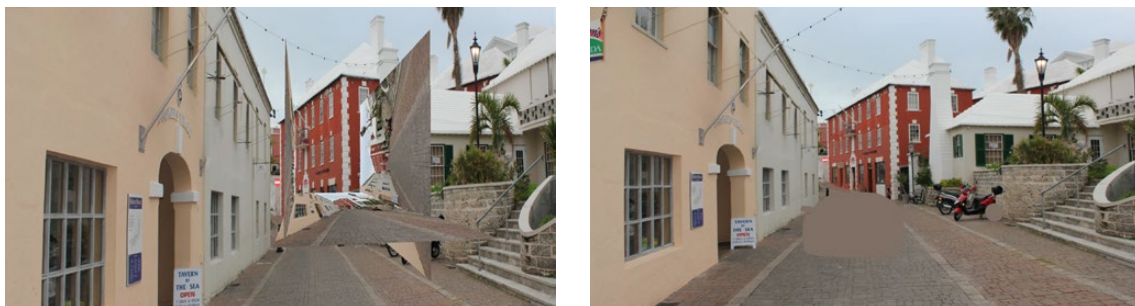
TIP: Projected textures can be allowed to slide across an object. If the object moves relative to the Projector 3D, or alternatively, by grouping the two together with a Merge3D, they can be moved as one and the texture will remain locked to the object.

In the following section of a much larger composition, an image (the Loader1 node) is projected into 3D space by mapping it onto five planes (Shape3D nodes renamed ground, LeftWall, RightWall, Building, and Background), which are positioned as necessary within a Merge3D node to apply reflections onto a 3D car to be composited into that scene.



Excerpt of a composition that's projecting an image of a street scene into 3D space

The output of the Merge3D node used to assemble those planes into a scene is then fed to a UV Map node, which in conjunction with a Camera3D node correctly projects all of these planes into 3D space so they appear as they would through that camera in the scene. Prior to this UVMap projection, you can see the planes arranged in space at left, where each plane has the scene texture mapped to it. At right is the image after the UVMap projection, where you can see that the scene once again looks “normal,” with the exception of a car-shaped hole introduced to the scene.



Five planes positioning a street scene in 3D space in preparation for UV Projection (left), and the UV Map node being used to project these planes so they appear as through a camera in the scene (right)

However, this is now a 3D scene, ready for a digital car to be placed within it, receiving reflections and lighting and casting shadows into the scene as if it were there.



The new 3D scene casting reflections and lighting onto a 3D car, and receiving shadows caused by the car

Geometry

There are five nodes used for creating geometry in Fusion. These nodes can be used for a variety of purposes. For instance, the Image Plane 3D is primarily used to place image clips into a 3D scene, while the Shapes node can add additional building elements to a 3D set, and Text 3D can add three-dimensional motion graphics for title sequences and commercials. Although each node is covered in more detail in the “3D Nodes” chapter, a summary of the 3D creation nodes is provided below.

- **Cube 3D**

The Cube 3D creates a cube with six inputs that allow mapping of different textures to each of the cube's faces.

- **Image Plane 3D**

The Image Plane 3D is the basic node used to place a 2D image into a 3D scene with an automatically scaled plane.

- **Shape 3D**

This node includes several basic primitive shapes for assembling a 3D scene. It can create planes, cubes, spheres, cylinders, cones, and toruses.

- **Text 3D**

The Text 3D is a 3D version of the Text+ node. This version supports beveling and extrusion but does not have support for the multi-layered shading model available from Text+.

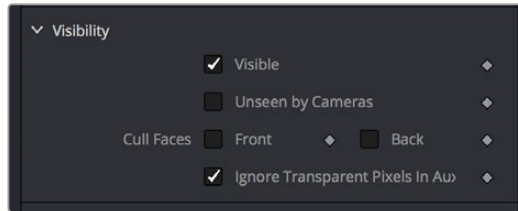
- **Particles**

When a pRender node is connected to a 3D view, it will export its particles into the 3D environment. The particles are then rendered using the Renderer3D instead of the Particle renderer.

For more information, see *Chapter 114, “Particle Nodes,”* in the DaVinci Resolve Reference Manual or *Chapter 54* in the Fusion Reference Manual.

Common Visibility Parameters

Visibility parameters are found in the Controls tab of most 3D geometry-producing nodes, exposed via a disclosure control. These parameters let you control object visibility in the viewers and in the final render.



A 3D geometry node's visibility parameters

- **Visible**

If the Visibility checkbox is not selected, the object will not be visible in a viewer, nor will it be rendered into the output image by a `Renderer3D`. A non-visible object does not cast shadows. This is usually enabled by default, so objects that you create are visible in both the viewers and final renders.

- **Unseen by Cameras**

If the Unseen by Cameras checkbox is selected, the object will be visible in the viewers but invisible when viewing the scene through a camera, so the object will not be rendered into the output image by a `Renderer3D`. Shadows cast by an Unseen object will still be visible.

- **Cull Front Face/Back Face**

Use these options to cull (exclude) rendering of certain polygons in the geometry. If Cull Back Face is selected, all polygons with normals pointing away from the view will not be rendered and will not cast shadows. If Cull Front Face is selected, all polygons with normals pointing away from the view will likewise be excluded. Selecting both checkboxes has the same effect as deselecting the Visible checkbox.

- **Ignore Transparent Pixels in Aux Channels**

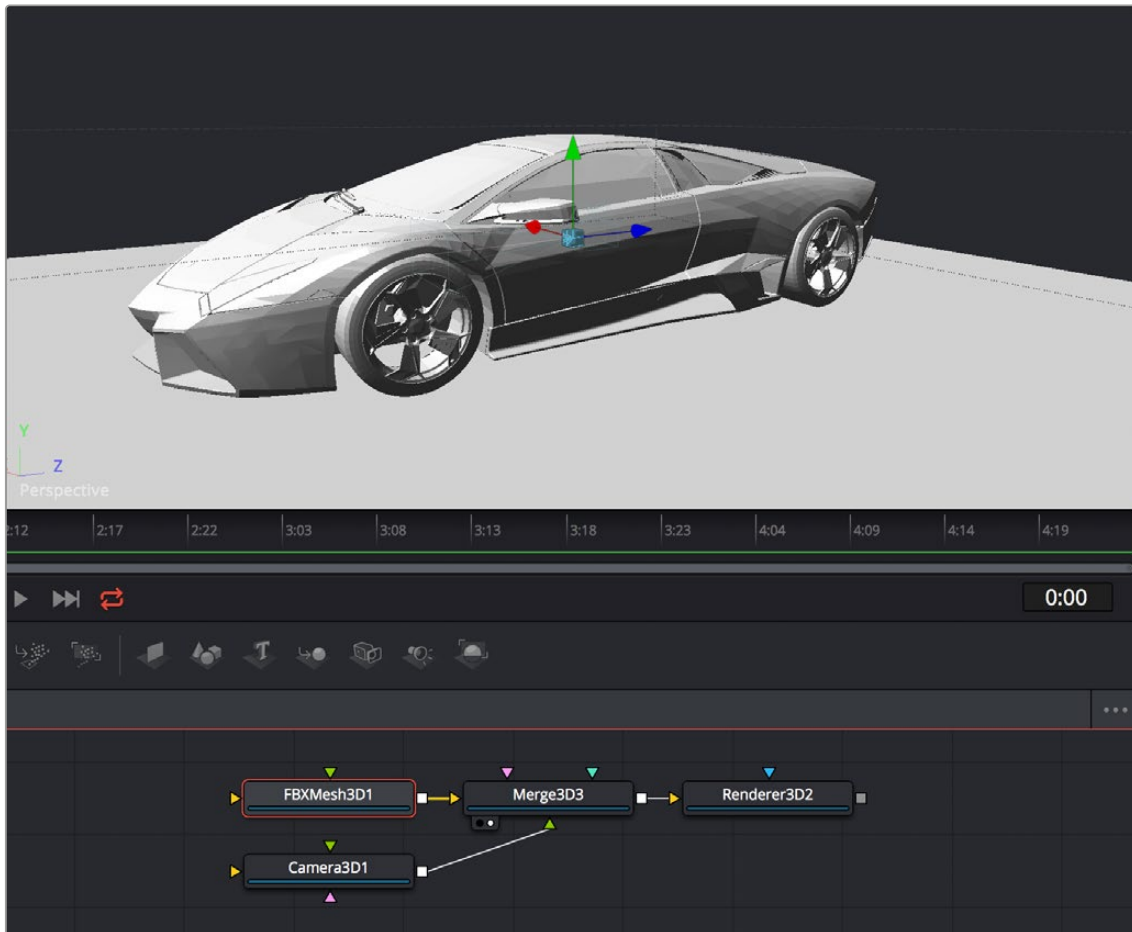
For any piece of geometry, the `Renderer3D` rejects transparent pixels in the auxiliary image channels. The reason this is the default is to prevent aux channels (e.g., normals, Z-channel, UVs) from filling in where there should be blank space or full transparency. For example, suppose in post you want to add some fog to the rendered image. If you had fully transparent geometry in the foreground affecting the Z-channel, you would get incorrect fog rendering. By deselecting this checkbox, the transparency will not be considered and all the aux channels will be filled for all the pixels. This could be useful if you wanted to replace texture on a 3D element that is fully transparent in certain areas with a texture that is transparent in different areas; it would be useful to have the whole object set aux channels (in particular UVs).

Adding FBX Models

The Filmbox FBX format is a scene interchange format that facilitates moving 3D scene information from one application to another. Fusion's FBX format support extends model import support to other 3D files such as Collada and OBJ.

Importing an FBX Scene

To import an entire FBX scene, you add an `FBXMesh3D` node to your node tree. After being prompted to choose a scene or object file, Fusion imports it to create a composition with the same lights, cameras, materials, and geometry found in an FBX file.



An imported model, via the FBXMesh3D node

FBX Scene Import Dialog

The FBX Mesh node is used to import mesh geometry from an FBX file. The first texture applied to a mesh will also be imported, if available.

Since different 3D applications use different units to measure their 3D scenes, the imported geometry may be enormous compared to the rest of the scene, because Fusion treats its scale of measurement as equal to its own system. For example, if your 3D application defaults to using millimeters as its scale, an object that was 100 millimeters in size will import as a massive 100 units.

You can use the Size slider in the FBX Mesh Inspector parameters to reduce the scale of such files to something that matches Fusion's 3D scene.

FBX Exporter

You can export a 3D scene from Fusion to other 3D packages using the FBX Exporter node. On render, it saves geometry, cameras lights, and animation into different file formats such as .dae or .fbx. The animation data can be included in one file, or it can be baked into sequential frames. Textures and materials cannot be exported.

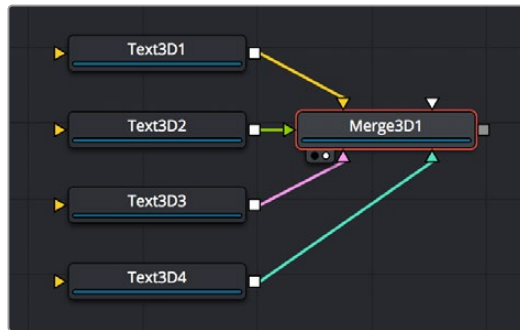
Using Text3D

The Text3D node is probably the most ubiquitous node employed by motion graphics artists looking to create titles and graphics from Fusion. It's a powerful node filled with enough controls to create nearly any text effect you might need, all in three dimensions. This section seeks to get you started quickly with what the Text3D node is capable of.

For more information, see *Chapter 89, "3D Nodes,"* in the DaVinci Resolve Reference Manual or *Chapter 29* in the Fusion Reference Manual.

Assembling Text Objects

Each Text3D node is a self-contained scene within which each character of text is an individual object. Because of this, the ideal way to combine numerous text objects that you might want to animate or style independently from one another is to connect as many Text3D objects as you want to be able to independently animate or style to one or more Merge3D nodes.

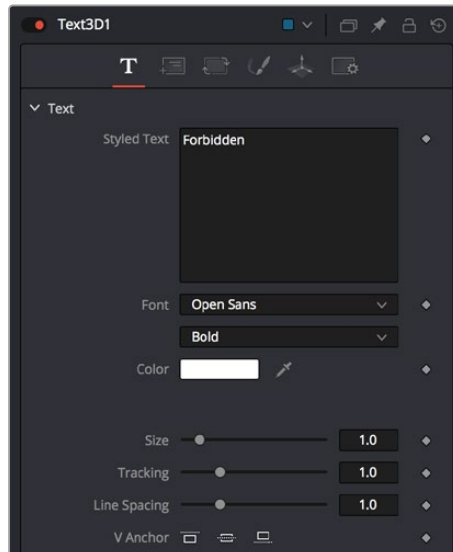


Merging multiple text objects to create an intricately styled scene

TIP: If you click the Text icon in the toolbar to create a Text3D node, and then you click it again while the Text3D node you just created is selected, a Merge3D node is automatically created and selected to connect the two. If you keep clicking the Text icon, more Text3D nodes will be added to the same selected Merge3D node.

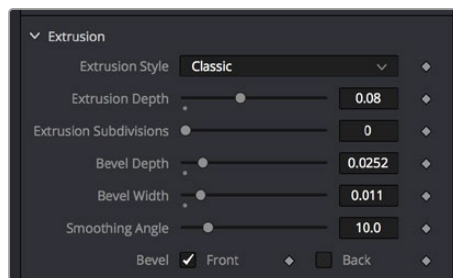
Entering Text

When you select a Text3D node and open the Inspector, the Text tab shows a “Styled Text” text entry field at the very top into which you can type the text you want to appear onscreen. Below, a set of overall styling parameters are available to set the Font, Color, Size, Tracking, and so on. All styling you do in this tab affects the entire set of text at once, which is why you need multiple text objects if you want differently styled words in the same scene.



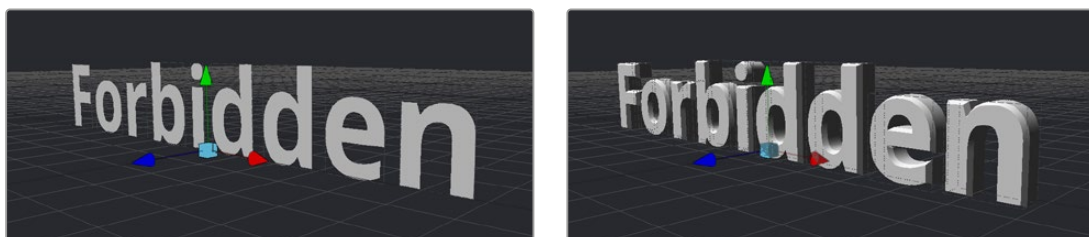
The text entry and styling parameters in the Text tab

Near the bottom of the Text tab are the Extrusion parameters, available within a disclosure control.



The Extrusion parameters near the bottom of the Text tab

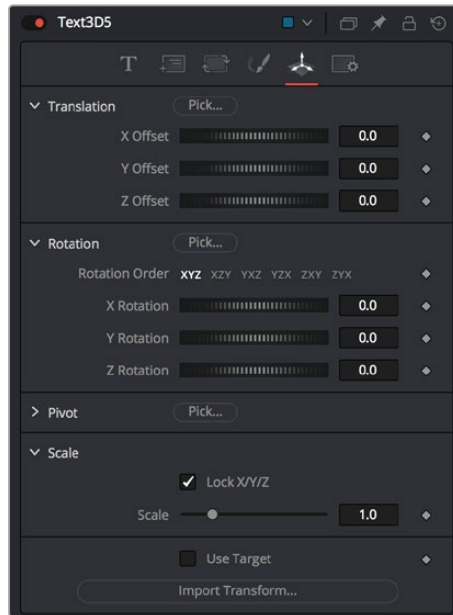
By default, all text created with the Text3D node is flat, but you can use the Extrusion Style, Extrusion Depth, and various Bevel parameters to give your text objects thickness.



Unextruded text (left), and Extruded text (right)

Positioning and Transforming Text

By default, every new Text3D node is positioned at 0, 0, 0, so when you add multiple Text3D nodes, they're all in the same place. Fortunately, every Text3D node has built-in transform controls in the Transform tab.



Text3D nodes also have Transform parameters built-in

Additionally, selecting a Text3D node exposes all the onscreen transform controls discussed elsewhere in this chapter. Using these controls, you can position and animate each text object independently.



Repositioned text objects to create a title sequence

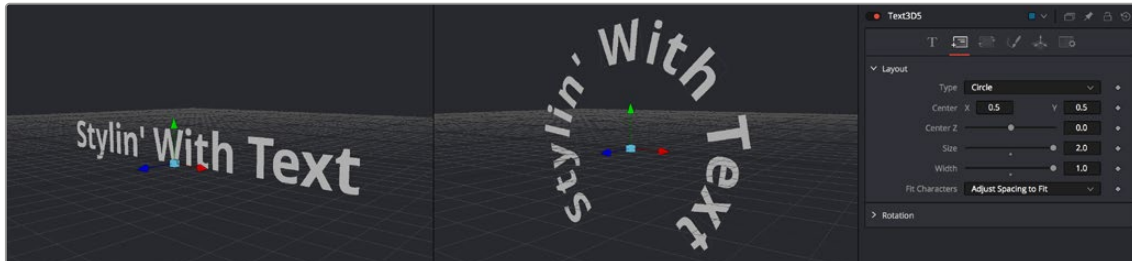
Combining Text3D nodes using Merge3D nodes doesn't just create a scene; it also enables you to transform your text objects either singly or in groups:

Selecting an individual Text3D node or piece of text in the viewer lets you move that one text object around by itself, independently of other objects in the scene.

Selecting a Merge3D node exposes a transform control that affects all objects connected to that Merge3D node at once, letting you transform the entire scene.

Layout Parameters

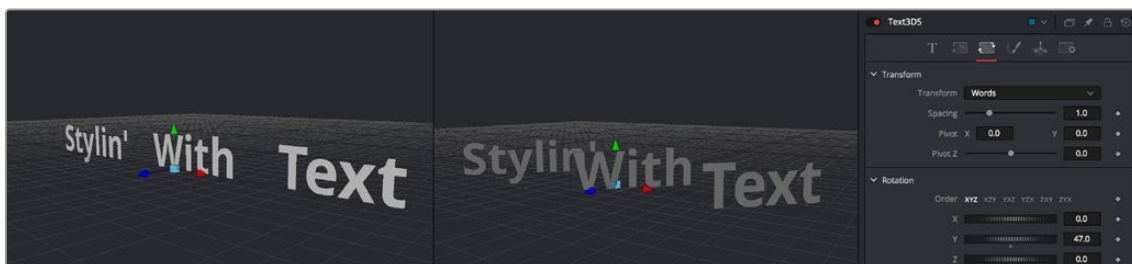
The Layout tab presents parameters you can use to choose how text is drawn: on a straight line, a frame, a circle, or a custom spline path, along with contextual parameters that change depending on which layout you've selected (all of which can be animated).



Text using two different layouts

“Sub” Transforms

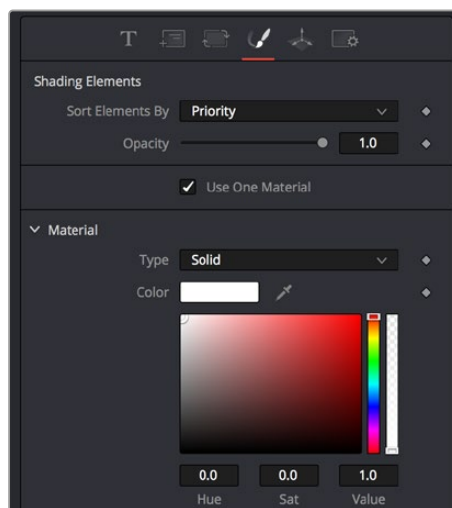
Another Transform tab (which the documentation has dubbed the “Sub” Transform tab) lets you apply a separate level of transform to either characters, words, or lines of text, which lets you create even more layout variations. For example, choosing to Transform by Words lets you change the spacing between words, rotate each word, and so on. You can apply simultaneous transforms to characters, words, and lines, so you can use all these capabilities at once if you really need to go for it. And, of course, all these parameters are animatable.



Transforming individual words in two different ways

Shading

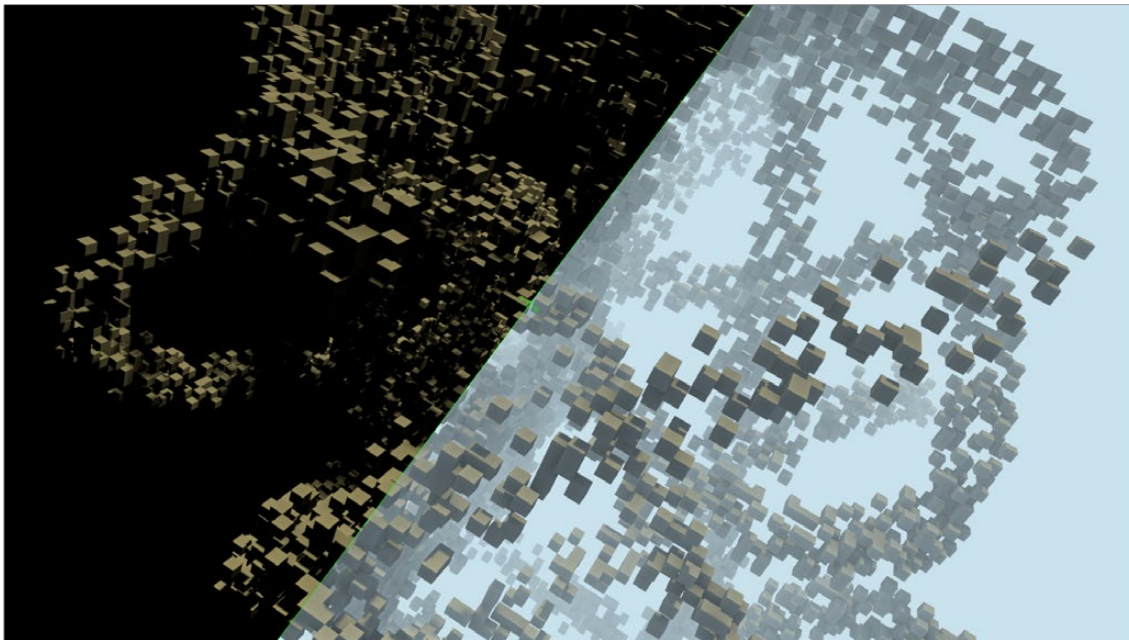
The Shading tab lets you shade or texture a text object using standard Material controls.



Shading controls for text objects

Fog 3D and Soft Clipping

The Fog3D node helps to create atmospheric depth cues.



Split screen with and without fog

The Fog3D node works well with depth of field and antialiasing supported by the OpenGL renderer. Since it is not a post-processing node (like the VolumeFog node found in the Nodes > Position menu or Fog node in Nodes > Deep Pixel), it does not need additional channels like Position or Z-channel color. Furthermore, it supports transparent objects.

The SoftClip node uses the distance of a pixel from the viewpoint to affect opacity, allowing objects to gradually fade away when too close to the camera. This prevents objects from “popping off” should the camera pass through them. This is especially useful with particles that the camera may be passing through.

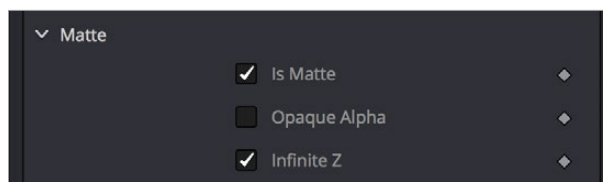
Geometry nodes such as the Shape3D node use a Matte Objects checkbox to enable masking out parts of the 3D scene. Effectively, everything that falls behind a matte object doesn’t get rendered. However, matte objects can contribute information into the Z-channel and the Object ID channel, leaving all other channels at their default values. They do not remove or change any geometry; they can be thought of as a 3D garbage matte for the renderer.



Circle shape used as a Matte object to see the floor

Matte Object Parameters

Opening the Matte disclosure control reveals the Is Matte option, which when turned on enables two more options.



Matte parameters in the Shape3D node; enabling Is Matte reveals additional options

- **Is Matte**

Located in the Controls tab for the geometry, this is the main checkbox for matte objects. When enabled, objects whose pixels fall behind the matte object's pixels in Z do not get rendered.

- **Opaque Alpha**

When the Is Matte checkbox is enabled, the Opaque Alpha checkbox is displayed. Enabling this checkbox sets the alpha value of the matte object to 1. Otherwise the alpha, like the RGB, will be 0.

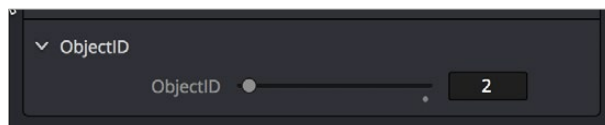
- **Infinite Z**

When the Is Matte checkbox is enabled, the Infinite Z checkbox is displayed. Enabling this checkbox sets the value in the Z-channel to infinite. Otherwise, the mesh will contribute normally to the Z-channel.

Matte objects cannot be selected in the viewer unless you right-click in the viewer and choose 3D Options > Show Matte Objects in the contextual menu. However, it's always possible to select the matte object by selecting its node in the node tree.

Material and Object IDs

Most nodes in Fusion that support effect masking can use Object ID and Material ID auxiliary channels to generate a mask. The parameters used to accomplish this are found in the Common Controls tab of each node.



Material ID parameters in a Shape3D node's Inspector controls

The Material ID is a value assigned to identify what material is used on an object. The Object ID is roughly comparable to the Material ID, except it identifies objects and not materials.

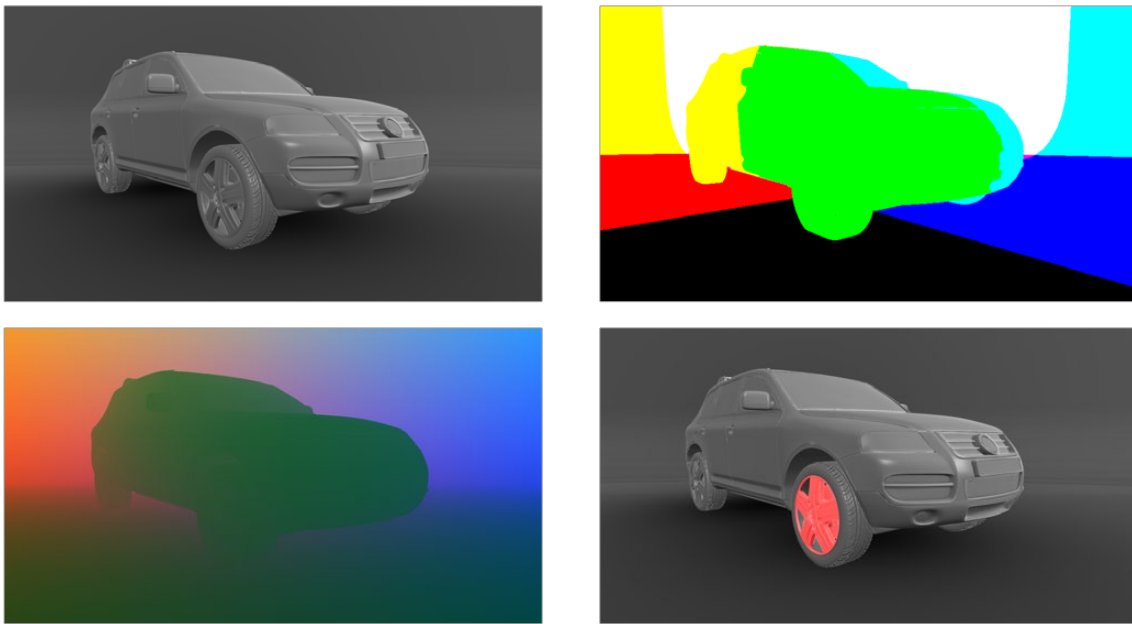
Both the Object ID and Material ID are assigned automatically in numerical order, beginning with 1. It is possible to set the IDs to the same value for multiple objects or materials even if they are different. Override 3D offers an easy way to change the IDs for several objects. The Renderer will write the assigned values into the frame buffers during rendering, when the output channel options for these buffers are enabled. It is possible to use a value range from 0 to 65534. Empty pixels have an ID of 0, so although it is possible to assign a value of 0 manually to an object or material, it is not advisable because a value of 0 tells Fusion to set an unused ID when it renders.



Object ID for ground plane and object set to the same numeric value

World Position Pass

The World Position Pass, or WPP, is a render pass generated from 3D applications. Each pixel is assigned the XYZ position where the pixel was generated in the world coordinates. So if the face from which the pixel was derived in the scene sits at (0,0,0), the resulting pixel will have a Position value of (0,0,0). If we visualize this as RGB, the pixel will be black. If a face sits at (1,0,0) in the original scene, the resulting RGB pixel will be red. Due to the huge range of possible positions in a typical 3D scene, and 7/8 of those possible positions containing negative coordinates, the Position channel is always rendered in 32-bit float.



A World Position Pass rendering of a scene with its center at (0,0,0) The actual image is on the left

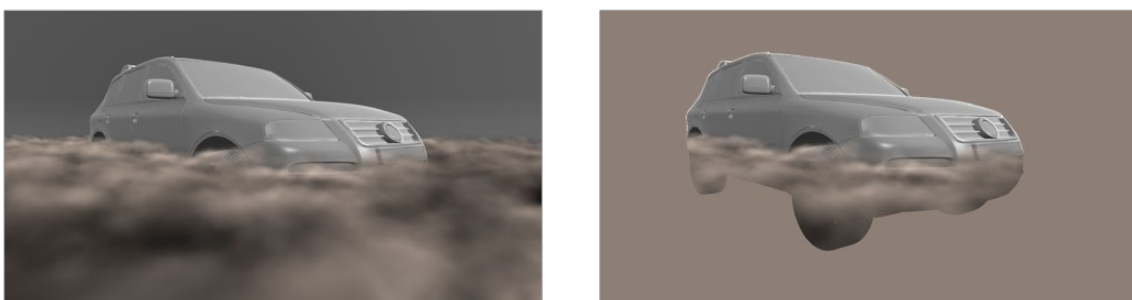
3D Scene Input

Nodes that utilize the World Position channel are located under the Position category. VolumeFog and Z to WorldPos require a camera input matching the camera that rendered the Position channels, which can either be a Camera3D or a 3D scene containing a camera. Just as in the Renderer3D, you can choose which camera to use if more than one are in the scene. The VolumeFog can render without a camera input from the Node Editor if the world space Camera Position inputs are set to the correct value. VolumeMask does not use a camera input. Nodes that support the World Position Pass, located under the Position category, offer a Scene input, which can be either a 3D Camera or a 3D scene containing a camera.

There are three Position nodes that can take advantage of World Position Pass data.

- Nodes > Position > Volume Fog
- Nodes > Position > Volume Mask
- Nodes > Position > Z to World
- The “Dark Box”

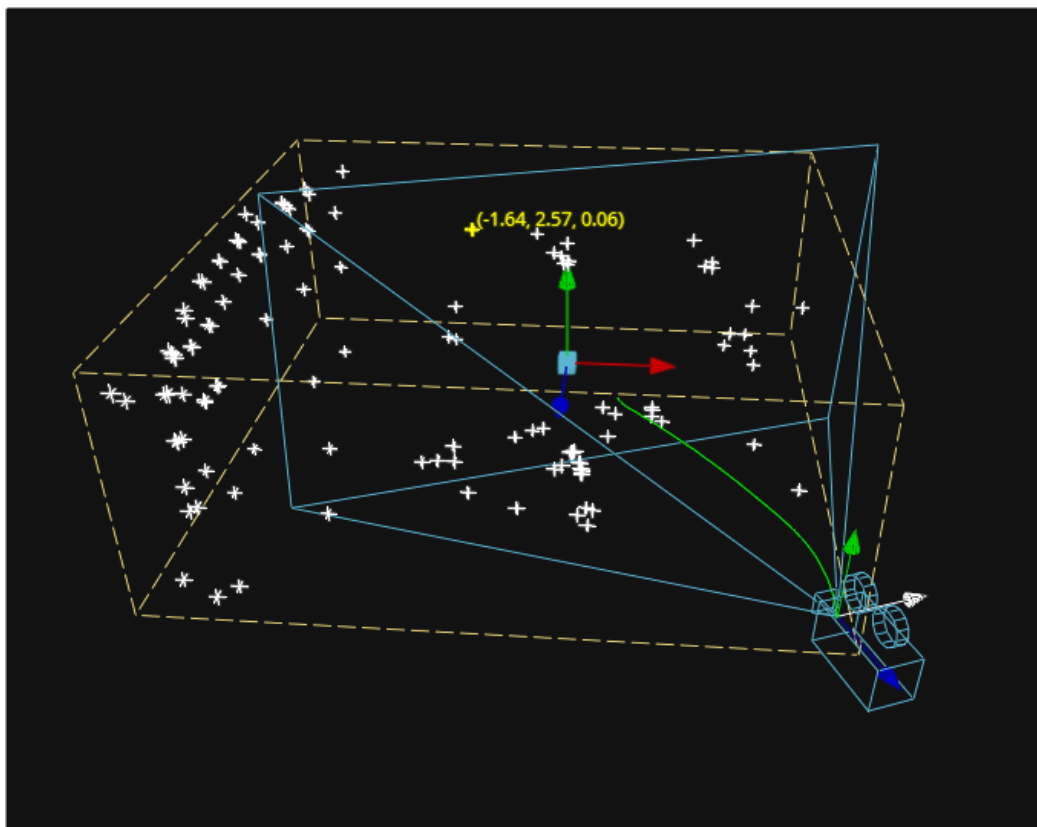
Empty regions of the render will have the Position channel incorrectly initialized to (0,0,0). To get the correct Position data, add a bounding sphere or box to your scene to create distant values and allow the Position nodes to render correctly.



Without a bounding mesh to generate Position values, the fog fills in the background incorrectly

Point Clouds

The Point Cloud node is designed to work with locator clouds generated from 3D tracking software. 3D camera tracking software, such as SynthEyes and PF Track, will often generate hundreds or even thousands of tracking points. Seeing these points in the scene and referencing their position in 3D and screen space is important to assist with lining up live action and CG, but bringing each point in as an individual Locator3D would impact performance dramatically and clutter the node tree.



Point cloud in the viewer

The Point Cloud node can import point clouds written into scene files from match moving or 3D scanning software.

To import a point cloud, do the following:

- 1 Add the PointCloud3D node to your composition.
- 2 Click the Import Point Cloud button in the Control panel.
- 3 Browse to the scene file and select a cloud to import from the scene.

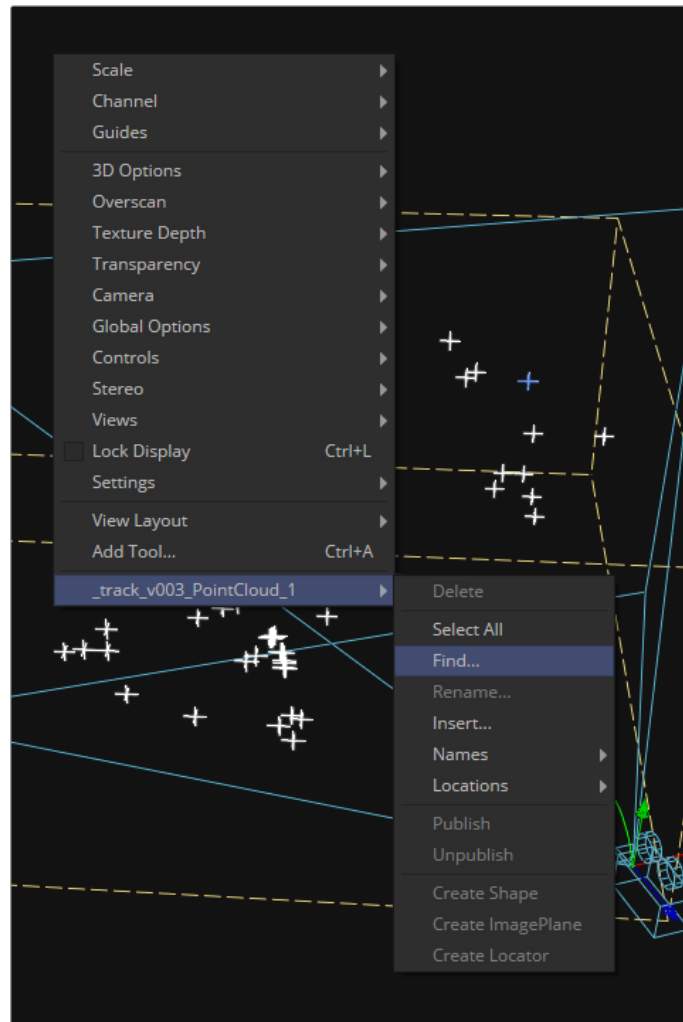
The entire point cloud is imported as one object, which is a significantly faster approach.

Finding, Naming, and Publishing Points

Many 3D trackers allow for the naming of individual tracking points, as well as setting tracking points on points of interest. The Point Cloud 3D will quickly find these points and publish them. A published point in the cloud can be used to drive the animation of other parameters.

To find a point in the point cloud, do the following:

- 1 Right-click anywhere within a viewer.
- 2 Choose Find from the Point Cloud's submenu in the contextual menu.
- 3 Type the name of the point and click OK.



Finding a point cloud using the viewer contextual menu

If a point that matches the name you entered is found, it will be selected in the point cloud and highlighted yellow.

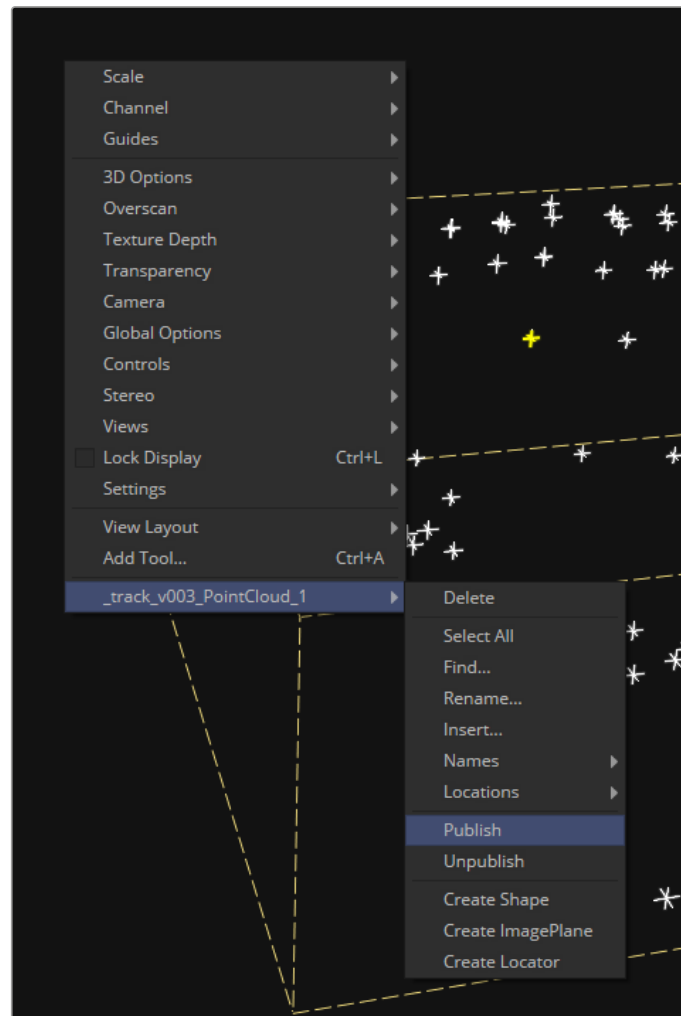
TIP: The Point Cloud Find function is a case-sensitive search. A point named “tracker15” will not be found if the search is for “Tracker15”.

Renaming a Point in the Cloud

You can use the Point Cloud contextual menu to rename a selected point. This works only for a single point. A group of points cannot be renamed.

Publishing a Point

If you want to use a point's XYZ positions for connections to other controls in the scene, you can publish the point. This is useful for connecting objects to the motion of an individual tracker. To publish a point, right-click it and choose Publish from the contextual menu.



Publishing a point using the viewer contextual menu